

Tribhuvan University

Faculty of Humanities and Social Sciences

Masters of Computer Application (MCA)

1st Semester

Advanced Database Management System(MCA-503)

Unit-2: Query Processing– [7 Hrs.]

Instructor

Tekendra Nath Yogi (M.SC CSIT TU)

Tekendranath@gmail.com

Contents

- **Unit2: Query Processing– [7 Hrs.]**
 - 2.1. Translating SQL Queries into Relational Algebra and Other Operators;
 - 2.2. Algorithms for External Sorting;
 - 2.3. Algorithm for SELECT operation;
 - 2.4. Implementing the JOIN operation;
 - 2.5. Algorithms for PROJECT and SET Operations;
 - 2.6. Implementing Aggregate Operations and Different types of JOINS;
 - 2.7. Combining Operations Using Pipelining;
 - 2.8. Parallel Algorithms for Query Processing.

Introduction to Query Processing

- Database queries are expressed in high-level, declarative languages like SQL.
 - Which specify what result is needed, not how to get it.
 - Suitable for human use, but is ill suited to be the system's internal representation of a query.
 - A more useful internal representation is one based on the extended relational algebra.
 - Variety of methods for computing the answer – several different
 - Queries for a interaction with database,
 - Extended relational algebra expressions for a query,
 - Ways to evaluate relational-algebra expressions –
 - Access Path, Access Algorithms.

Cont'd...

- Thus, Query processing is necessary for queries that are specified in a high-level query language.
- **Query processing** refers to the process of
 - **Translating,**
 - **Optimizing,** and
 - **Executing/evaluating**

high level database queries internally by the DBMS in order to efficiently retrieve and manipulate data from the database.

Cont'd...

■ Query Processing Steps:

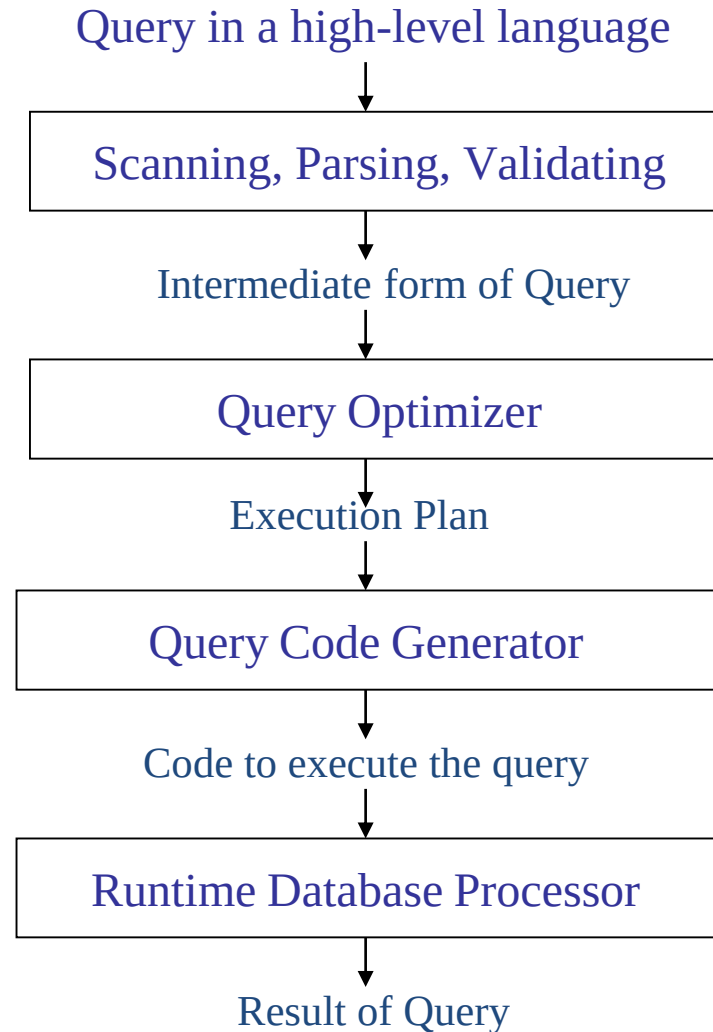


Figure: Typical steps when processing a high-level query

Cont'd...

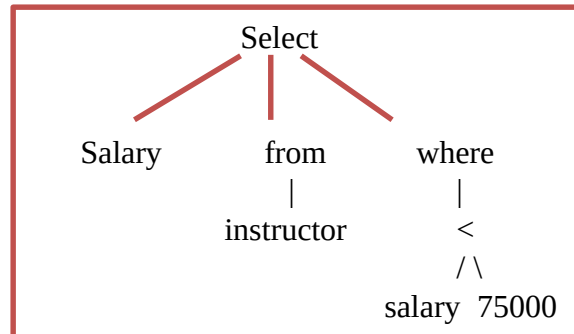
- **Scanning, Parsing, Validating:** A query expressed in a high-level query language such as SQL or OQL must first be **scanned**, **parsed**, and **validated**.
 - **Scanning:** The **scanner** identifies the query tokens such as SQL keywords, attribute names, and relation names that appear in the text of the query,
 - **Parsing:** The **parser** checks the query syntax to determine whether it is formulated according to the syntax rules (rules of grammar) of the query language.
 - **Validating:** The query must also be **validated** by checking that all attribute and relation names are valid and semantically meaningful names in the schema of the particular database being queried.

Cont'd...

- For example, Consider the query
select salary
from instructor
where salary < 75000;
- In Scanning, Parsing and Validation phase above query gets processed as follows:

Token	Type
SELECT	Keyword
salary	Attribute
FROM	Keyword
instructor	Relation (Table)
WHERE	Keyword
salary	Attribute
<	Operator
75000	Constant (Number)

Scanning



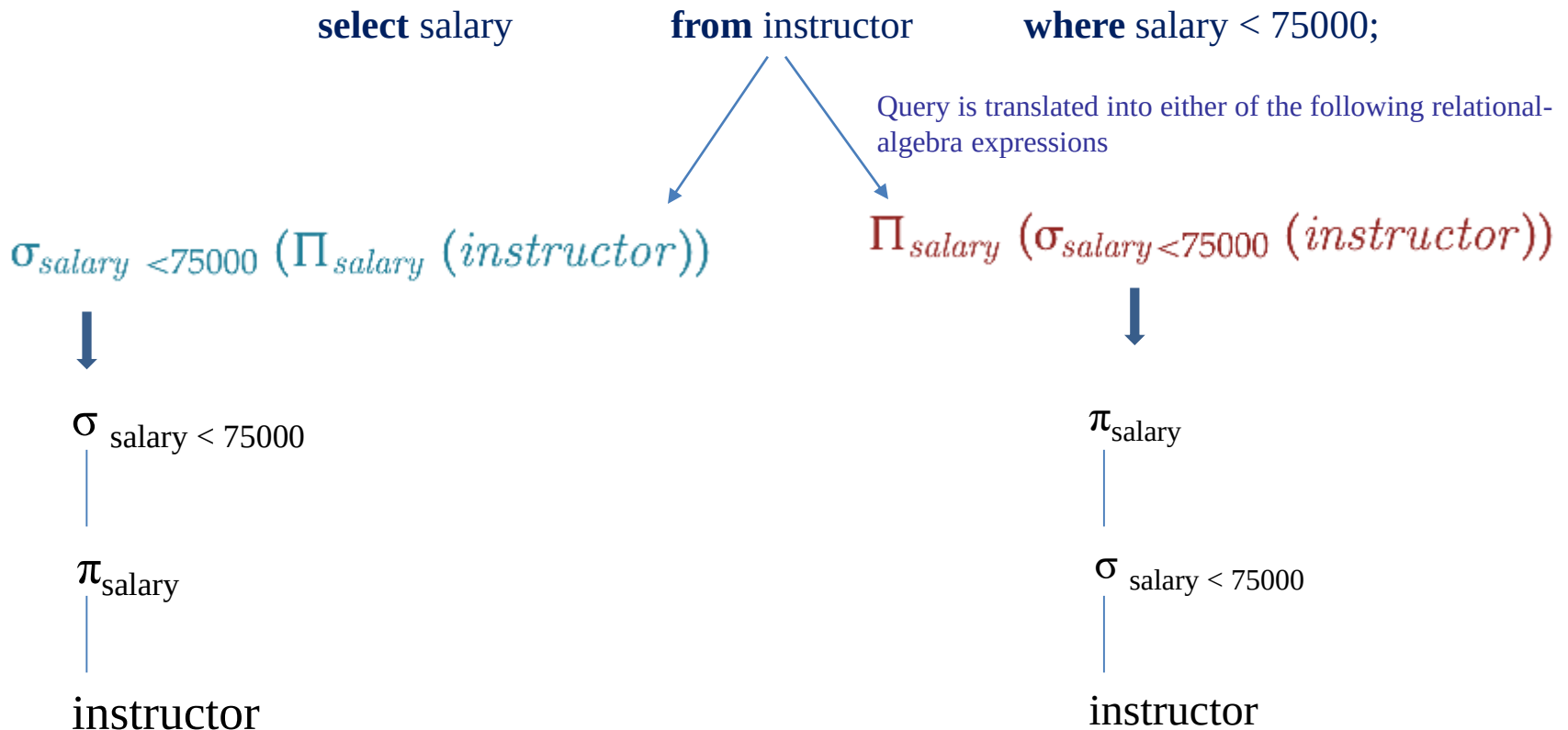
Parsing

Check	Result
Table instructor exists?	Yes
Attribute salary exists?	Yes
Data type of salary numeric?	Yes
Condition salary < 75000 valid?	Yes

Validating

Cont'd...

- An **internal representation** of the query (Query Tree/ Query Graph - DAG) is then created.
- For example,



Cont'd...

- **Query Optimization:** The DBMS must then devise an **execution strategy** or **query plan** for retrieving the results of the query from the database files.
 - A sequence of primitive operations that can be used to evaluate a query is called **execution strategy** or **query-evaluation plan**.
 - Annotate query tree with instructions specifying how to evaluate each operation.
 - That is, state the algorithm to be used for a specific operation , or the particular index or indices to use.

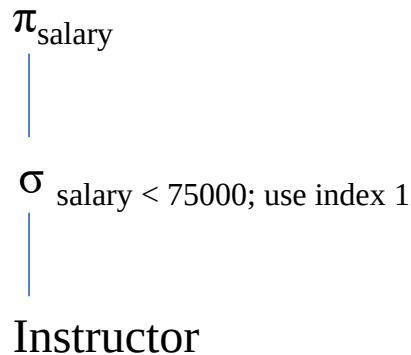


Figure: Query Evaluation Plan 1

OR

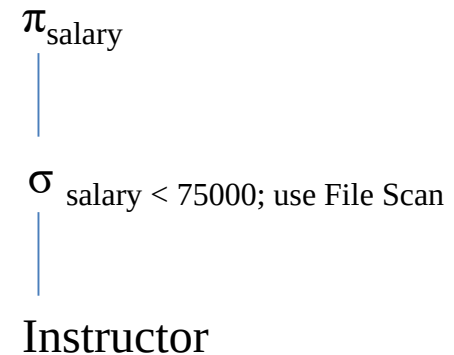


Figure: Query Evaluation Plan 2

Cont'd...

- The **query optimizer** module has the task of **query optimization** by producing a good execution plan.
 - A query has many possible execution strategies, and the process of choosing a suitable one for processing a query is known as **query optimization**.
 - There are two main techniques of query optimization:
 - **Heuristic Query Optimization:** Based on **heuristic** rules for ordering the operations in a query execution strategy.
 - **Cost Based Query Optimization:** Estimate the cost of different execution strategies and choose the execution plan that minimizes estimated cost.

Cont'd...

- **Query Code Generation:** The **code generator** generates the code to execute that plan
- **Execution :** The **runtime database processor** has the task of running (executing) the query code, whether in compiled or interpreted mode, to produce the query result.
 - If a runtime error results, an error message is generated by the runtime database processor.

2.1. Translating SQL Queries into Relational Algebra and Other Operators

- To Translate SQL Queries into relational algebra SQL Query is decomposed into **query blocks**.
 - Basic units that can be translated into the algebraic operators.
 - Contains single SELECT-FROM-WHERE expression
 - May contain GROUP BY and HAVING clauses
 - Nested queries within a query are identified as separate query blocks.
- Query blocks are then translated into the algebraic operators and optimized.
- Aggregate operators in the query (such as MAX, MIN, SUM, and COUNT) must be included in the extended algebra.

Translating SQL Queries (cont'd.)

- **Example:** Consider the following SQL query retrieves the names of employees who earn a salary that is greater than the highest salary in department 5.

```
SELECT Lname, Fname
FROM EMPLOYEE
WHERE Salary >
(
    SELECT MAX(Salary)
    FROM EMPLOYEE
    WHERE Dno = 5
);
```

The query includes a nested subquery and hence would be decomposed into two blocks.

- **Inner block:**

```
SELECT MAX(Salary)
FROM EMPLOYEE
WHERE Dno = 5
```

- **Outer block:**

```
SELECT Lname, Fname
FROM EMPLOYEE
WHERE Salary > c
```

Here, c represent the result of inner query

Translating SQL Queries (cont'd.)

- Example (cont'd.)

- Inner block translated into:

$$\mathcal{F}_{\text{MAX SALARY}}(\sigma_{\text{DNO}=5}(\text{EMPLOYEE}))$$

- Outer block translated into:

$$\pi_{\text{LNAME, FNAME}}(\sigma_{\text{SALARY} > C}(\text{EMPLOYEE}))$$

- Query optimizer chooses execution plan for each query block. For example,

C = MAX(SALARY)
|
 σ DNO = 5 ; File or index scan
|
EMPLOYEE

π LNAME, FNAME
|
 σ SALARY > C ; File or index scan
|
EMPLOYEE

Cont'd...

- **Additional Operators Semi-Join and Anti-Join: Semi-join**
 - Generally used for un-nesting EXISTS, IN, and ANY subqueries
 - Syntax: $T1.X \text{ **S** } T2.Y$ or $T1.X \bowtie T2.Y$
 - T1 is the left table and T2 is the right table of the semi-join
 - A row of T1 is returned as soon as T1.X finds a match with any value of T2.Y without searching for further matches

Cont'd...

- Additional Operators Semi-Join and Anti-Join: Semi-join Example:**

```
SELECT COUNT(*)
FROM DEPARTMENT D
WHERE D.Dnumber IN (
    SELECT E.Dno
    FROM EMPLOYEE E
    WHERE E.Salary > 200000
);
```



Un-nesting

```
SELECT COUNT(*)
FROM DEPARTMENT D, EMPLOYEE E
WHERE D.Dnumber = E.Dno AND E.Salary > 200000;
```

Relational Algebraic form of query:
 $\mathcal{F}$ COUNT(*)

(
DEPARTMENT $\bowtie_{D.Dnumber = E.Dno}$
 $\sigma_{E.Salary > 200000}$ (EMPLOYEE)
)



$\mathcal{F}$ COUNT(*)

|
 $\bowtie_{D.Dnumber = E.Dno}$; File scan/Index

DEPARTMENT

|
 $\sigma_{E.Salary > 200000}$
; File / Index Scan

EMPLOYEE

Query counts the number of departments that have employees who make more than 200,000 annually.

Cont'd...

■ Additional Operators Semi-Join and Anti-Join: Anti-join

- Used for un-nesting NOT IN, NOT EXISTS and ALL subqueries
- Syntax: $T1.x \neq T2.y$ or $T1.x \not\bowtie T2.y$
 - T1 is the left table and T2 is the right table of the anti-join
- A row of T1 is rejected as soon as T1.x finds a match with any value of T2.y
- A row of T1 is returned only if T1.x does not match with any value of T2.y

Cont'd...

■ Additional Operators Semi-Join and Anti-Join: Anti-join Example

```
SELECT COUNT(*)
FROM EMPLOYEE
WHERE EMPLOYEE.Dno NOT IN
    (
        SELECT DEPARTMENT.Dnumber
        FROM DEPARTMENT
        WHERE Zipcode = 30332
    );
```

 **Un-nesting**

```
SELECT COUNT(*)
FROM EMPLOYEE, DEPARTMENT
WHERE EMPLOYEE.Dno A= DEPARTMENT.Dnumber
AND Zipcode = 30332;
```

Relational algebraic form of Query:
 $\mathcal{F}$ COUNT(*)
(
EMPLOYEE $\bowtie_{Dno = Dnumber}$
 $\sigma_{Zipcode \neq 30332}$ (DEPARTMENT)
)



$\mathcal{F}$ COUNT(*)
|
 $\bowtie_{Dno = Dnumber}$; File or index scan
|
EMPLOYEE
|
 $\sigma_{Zipcode \neq 30332}$;
File or index Scan
|
DEPARTMENT

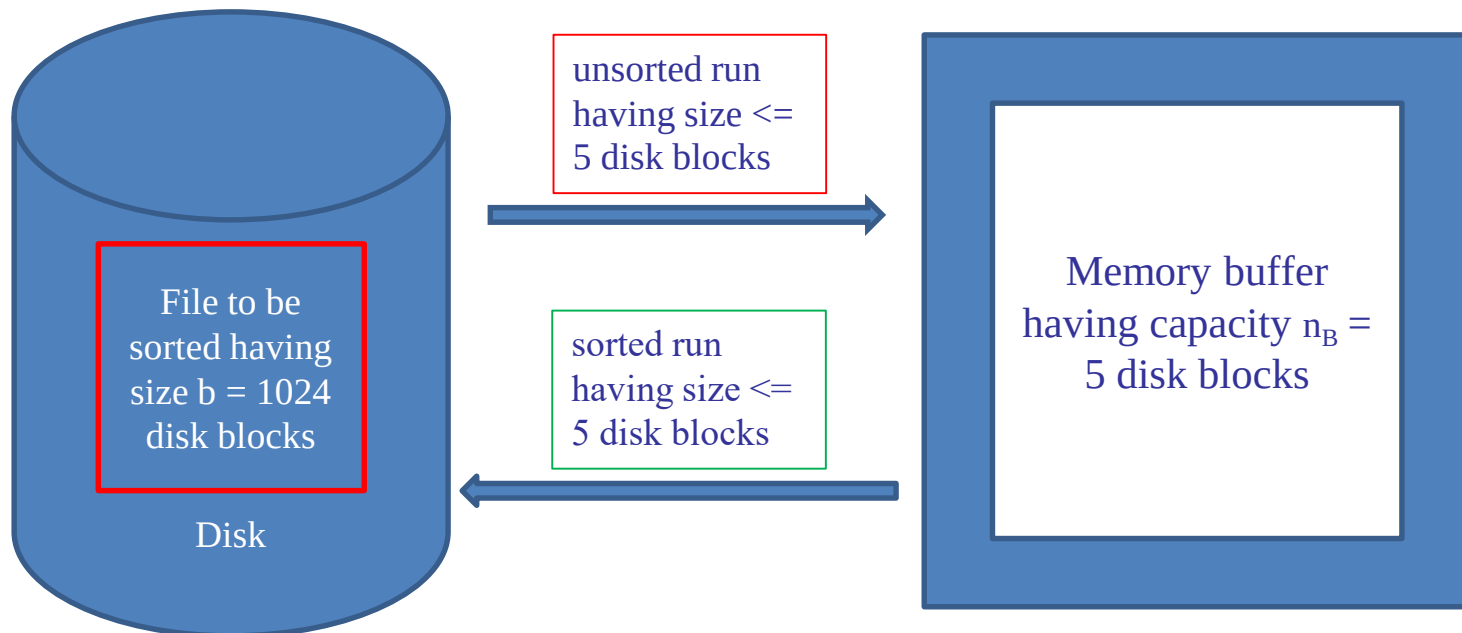
Query counts the number of employees who do not work in departments located in zip code 30332.

2.2 Algorithms for External Sorting

- Sorting is an often-used algorithm in query processing.
- Mainly used for queries containing operations like ORDER BY, JOIN, UNION, INTERSECTION, DISTINCT, etc.
- **External sorting:**
 - Sorting Algorithm suitable for large files stored on disk that do not fit entirely in main memory.
 - Uses **Sort-merge strategy** – by sorting smaller subfiles (runs) of the main file and then merges the sorted runs, creating larger sorted subfiles that are merged in turn.
 - Algorithm requires buffer space in main memory where the actual sorting and merging of the runs is performed.

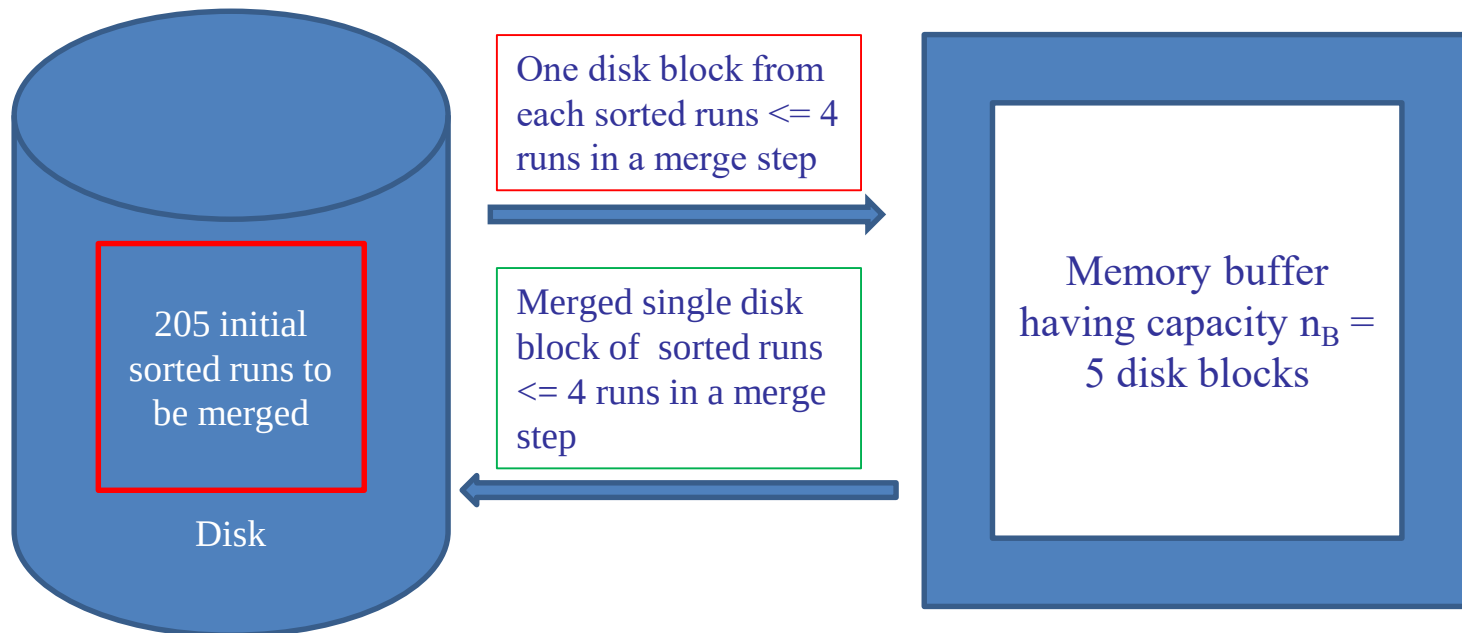
Cont'd...

- **Sort-merge algorithm:** In the **sorting phase**, runs of the file that can fit in the available buffer space are read into main memory, sorted using an internal sorting algorithm, and written back to disk as temporary sorted subfiles (or runs).
- Number of initial runs(n_R)= $\lceil (\text{number of file blocks } (b) / \text{available buffer space } (n_B)) \rceil$
 $= \lceil 1024 / 5 \rceil = 205$ sorted runs
- i.e., After sorting phase, 205 sorted runs are stored as temporary subfiles on disk.



Cont'd...

- **Sort-merge algorithm:** In the **merging phase**, the sorted runs are merged during one or more merge passes having one or more merge steps.
 - During each merge step, one buffer block is needed to hold one disk block from each of the sorted subfiles being merged, and one additional buffer is needed for containing one disk block of the merge result, which will produce a larger sorted file that is the result of merging several smaller sorted subfiles.

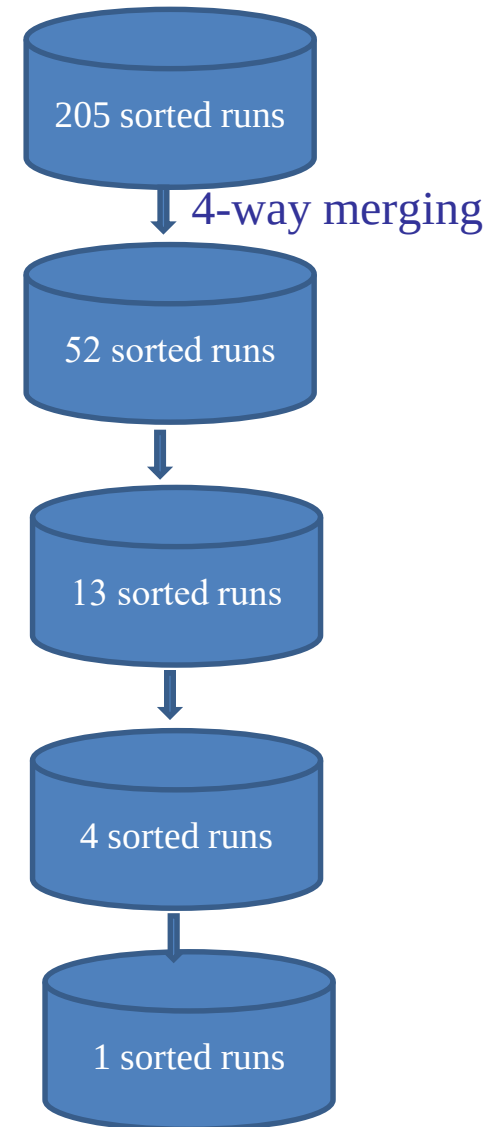


Cont'd...

- The **degree of merging (d_M)** is the number of sorted subfiles that can be merged in each merge step.

$$\begin{aligned}(d_M) &= \text{Min}((n_B - 1), n_R) \\ &= \text{Min}(5-1, 205) \\ &= 4 \text{ files}\end{aligned}$$

- **Number of merge passes** is $\lceil (\log_{d_M}(n_R)) \rceil$
 $= \lceil (\log_4(205)) \rceil$
 $= \lceil 3.84 \rceil$
 $= 4 \text{ passes}$
- Example, For $n_B = 5$, $d_M = 4$ (four-way merging), the 205 initial sorted runs would be merged 4 at a time in each step.



Cont'd...

```
set     $i \leftarrow 1$ ;  
        $j \leftarrow b$ ;           {size of the file in blocks}  
        $k \leftarrow n_B$ ;         {size of buffer in blocks}  
        $m \leftarrow \lceil (j/k) \rceil$ ; {number of subfiles- each fits in buffer}  
{Sorting Phase}  
while ( $i \leq m$ )  
do {  
    read next  $k$  blocks of the file into the buffer or if there are less than  $k$  blocks  
    remaining, then read in the remaining blocks;  
    sort the records in the buffer and write as a temporary subfile;  
     $i \leftarrow i + 1$ ;  
}  
{Merging Phase: merge subfiles until only 1 remains}  
set     $i \leftarrow 1$ ;  
        $p \leftarrow \lceil \log_{k-1} m \rceil$  { $p$  is the number of passes for the merging phase}  
        $j \leftarrow m$ ;  
while ( $i \leq p$ )  
do {  
     $n \leftarrow 1$ ;  
     $q \leftarrow \lceil j/(k-1) \rceil$ ; {number of subfiles to write in this pass}  
    while ( $n \leq q$ )  
    do {  
        read next  $k-1$  subfiles or remaining subfiles (from previous pass)  
        one block at a time;  
        merge and write as new subfile one block at a time;  
         $n \leftarrow n + 1$ ;  
    }  
     $j \leftarrow q$ ;  
     $i \leftarrow i + 1$ ;  
}
```

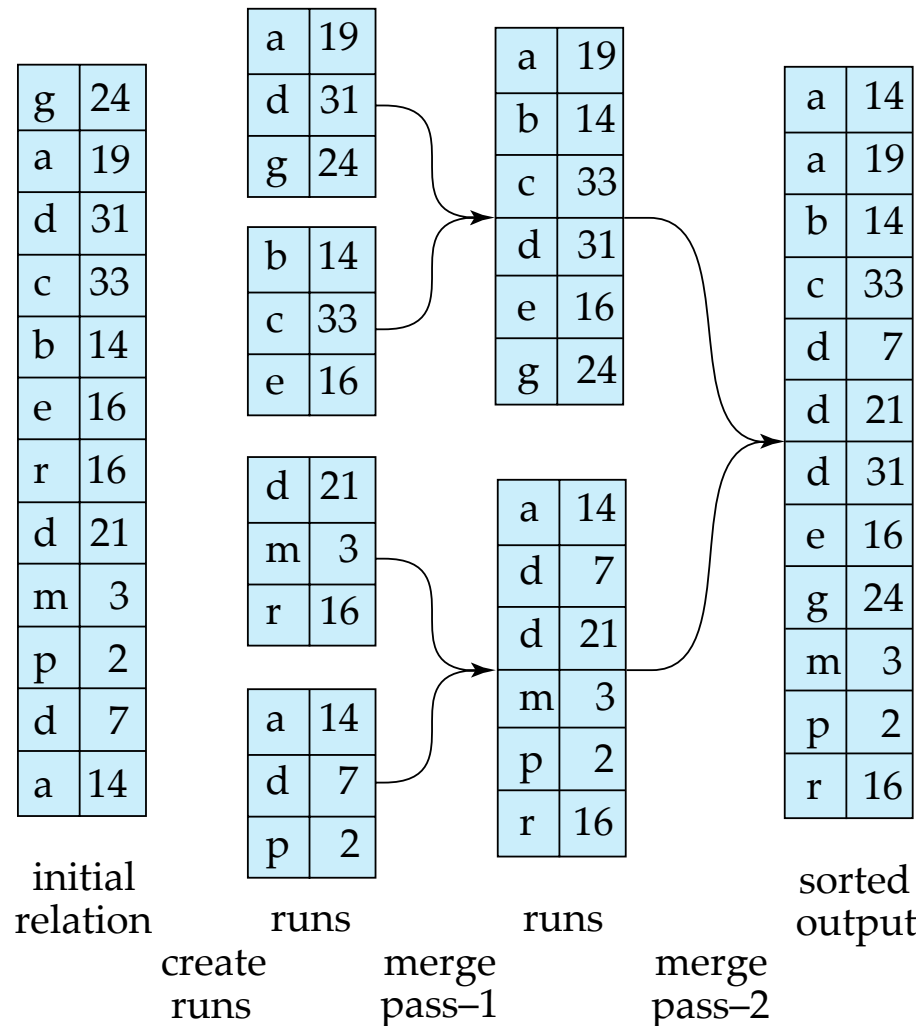
Figure: Outline of the sort-merge algorithm for external sorting

Cont'd...

- **Performance** of the sort-merge algorithm
 - Measured in number of disk block reads and writes before sorting is completed
 - = number of block accesses for the sorting phase + number of block accesses for the merging phase
 - $= (2 * b) + (\text{number of disk blocks is read and written} * \text{number of merge passes})$
 - $= (2 * b) + (2 * b * (\log_{dM} n_R))$

Cont'd...

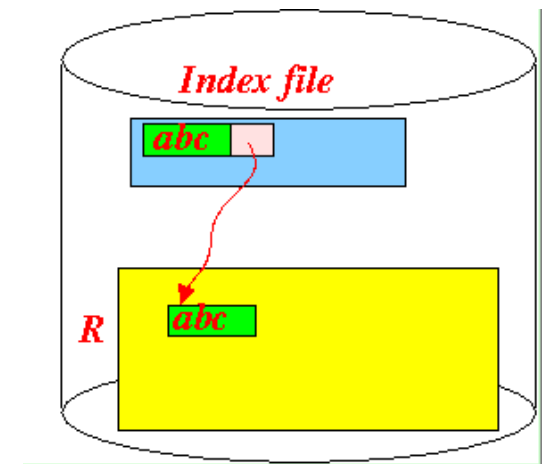
■ Example: External Sorting Using Sort-Merge



3.3 Algorithms for SELECT Operation

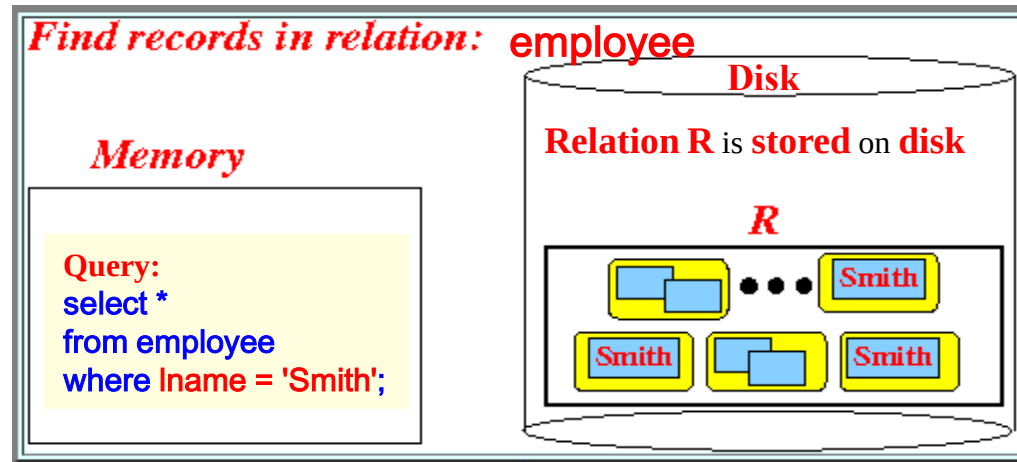
■ SELECT operation

- Search operation to locate records in a disk file that satisfy a certain condition.
- A number of search algorithms are possible for selecting records from a file. These algorithms are categorized into following two general categories:
 - **File scan:** scan the records of a file to search for and retrieve records that satisfy a selection condition.
 - **Index scan:** search algorithm involves the use of an index.

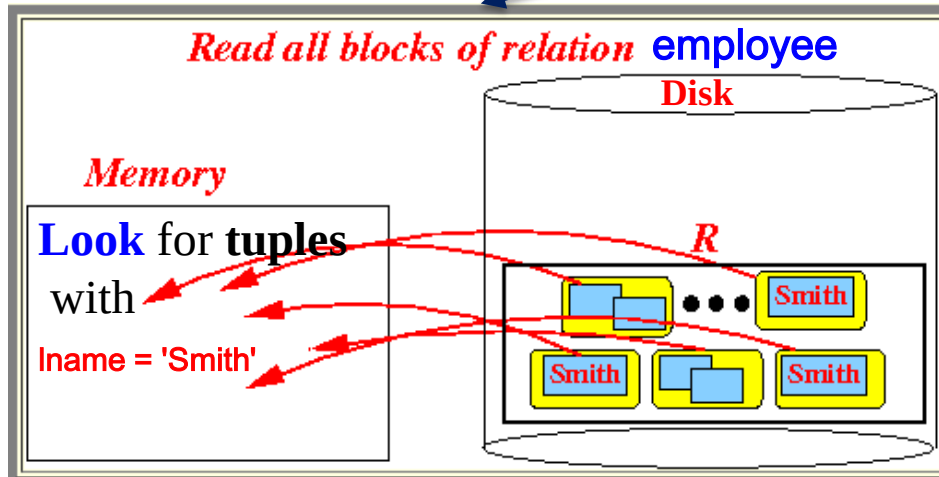


Cont'd...

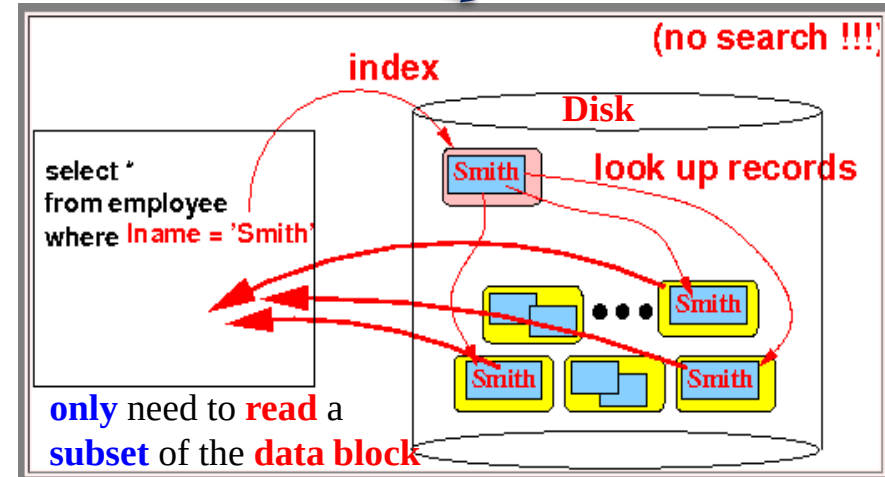
■ Example:



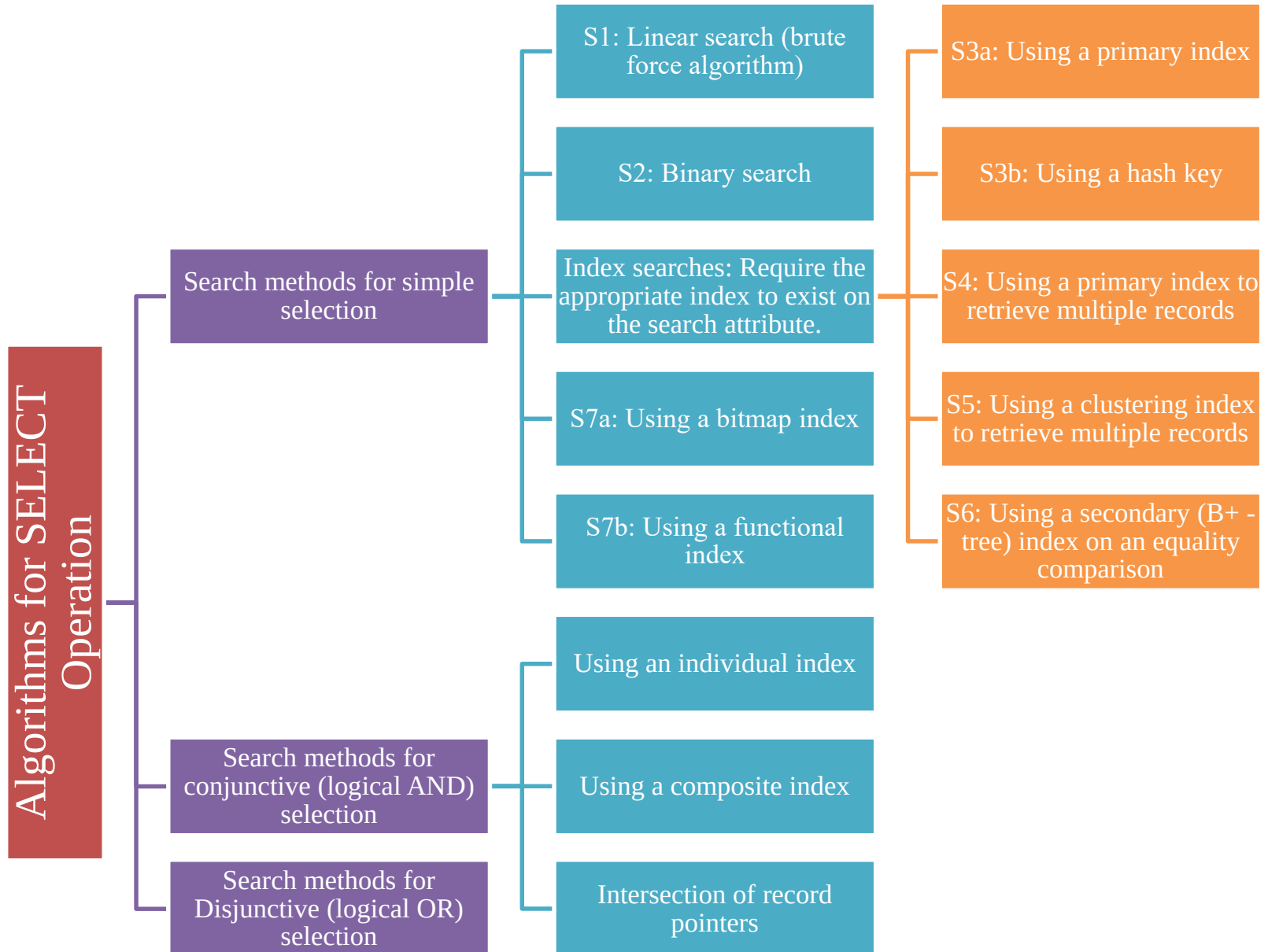
File scan



Index scan



Cont'd...



Cont'd...

■ S1 Linear search (brute force):

- Retrieve every record in the file, and test whether its attribute values satisfy the selection condition.
- For example, $\sigma_{SSN='123456789'}(EMPLOYEE)$
- For the query like above the DBMS can only check whether or not an access path exists on the attribute involved in the condition.
- If an access path (such as index or hash key or bitmap index or sorted file) exists, the method corresponding to that access path is used; otherwise, the brute force, **linear search** approach can be used.

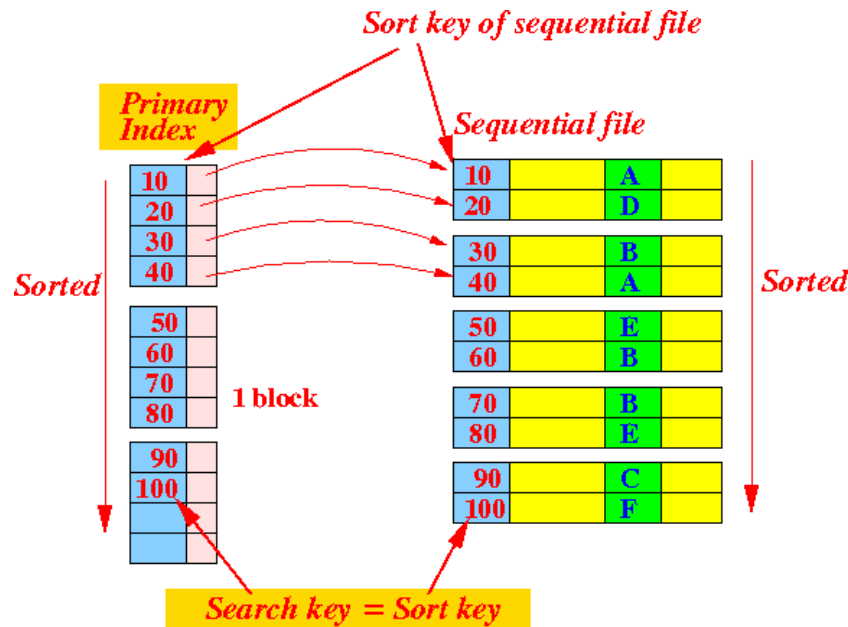
Cont'd...

■ S2 Binary search:

- If the selection condition involves an equality comparison on a **key attribute** on which the file is ordered, binary search can be used.
- For example, $\sigma_{SSN='123456789'}(EMPLOYEE)$
- Above query uses binary search if Ssn is the ordering attribute for the EMPLOYEE file.

Cont'd...

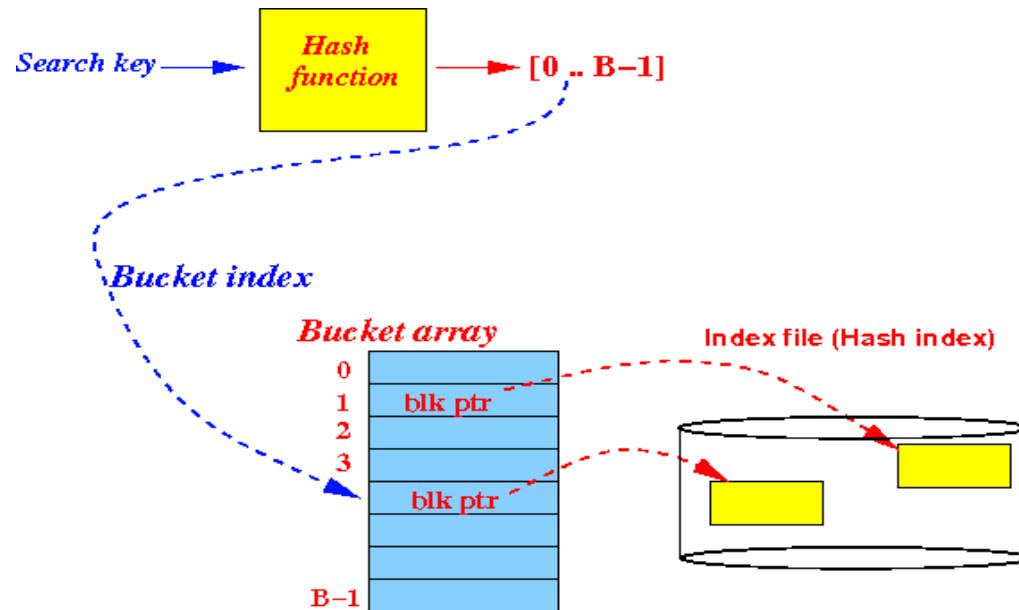
- **S3 (3a) Using a primary index to retrieve a single record:**
 - If the selection condition involves an equality comparison on a key attribute with a primary index, use the primary index to retrieve the record.
 - For example, $\sigma_{SSN='123456789'}(EMPLOYEE)$
 - Above query uses the primary index to retrieve the single record if the SSN is used as a primary index.



Cont'd...

■ S3 (3b) Using a hash key to retrieve a single record:

- If the selection condition involves an equality comparison on a key attribute with a hash key, use the hash key to retrieve the record.
- For example, $\sigma_{\text{SSN}='123456789'}(\text{EMPLOYEE})$
- Above query uses the hash index to retrieve the single record if the SSN is used as a hash index.



Cont'd...

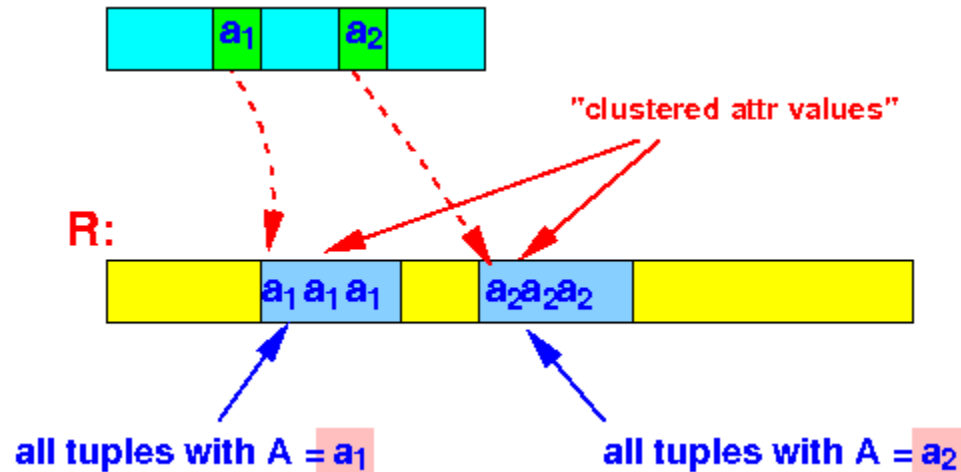
■ S4 Using a primary index to retrieve multiple records:

- If the comparison condition is $>$, $\geq$, $<$, or $\leq$ on a key field with a primary index, use the index to find the record satisfying the corresponding equality condition, then retrieve all subsequent records in the (ordered) file.
- For example, $\sigma_{\text{DNUMBER} > 5}(\text{DEPARTMENT})$
- Dnumber > 5 in the above query use the index to find the record satisfying the corresponding equality condition (Dnumber = 5); then retrieve all subsequent records in the (ordered) file.

Cont'd...

■ S5 Using a clustering index to retrieve multiple records:

- If the selection condition involves an equality comparison on a non-key attribute with a clustering index, use the clustering index to retrieve all the records satisfying the selection condition.
- For example, $\sigma_{DNO=5}(\text{EMPLOYEE})$
- Dno = 5 in the above query use the index to retrieve all the records satisfying the condition.



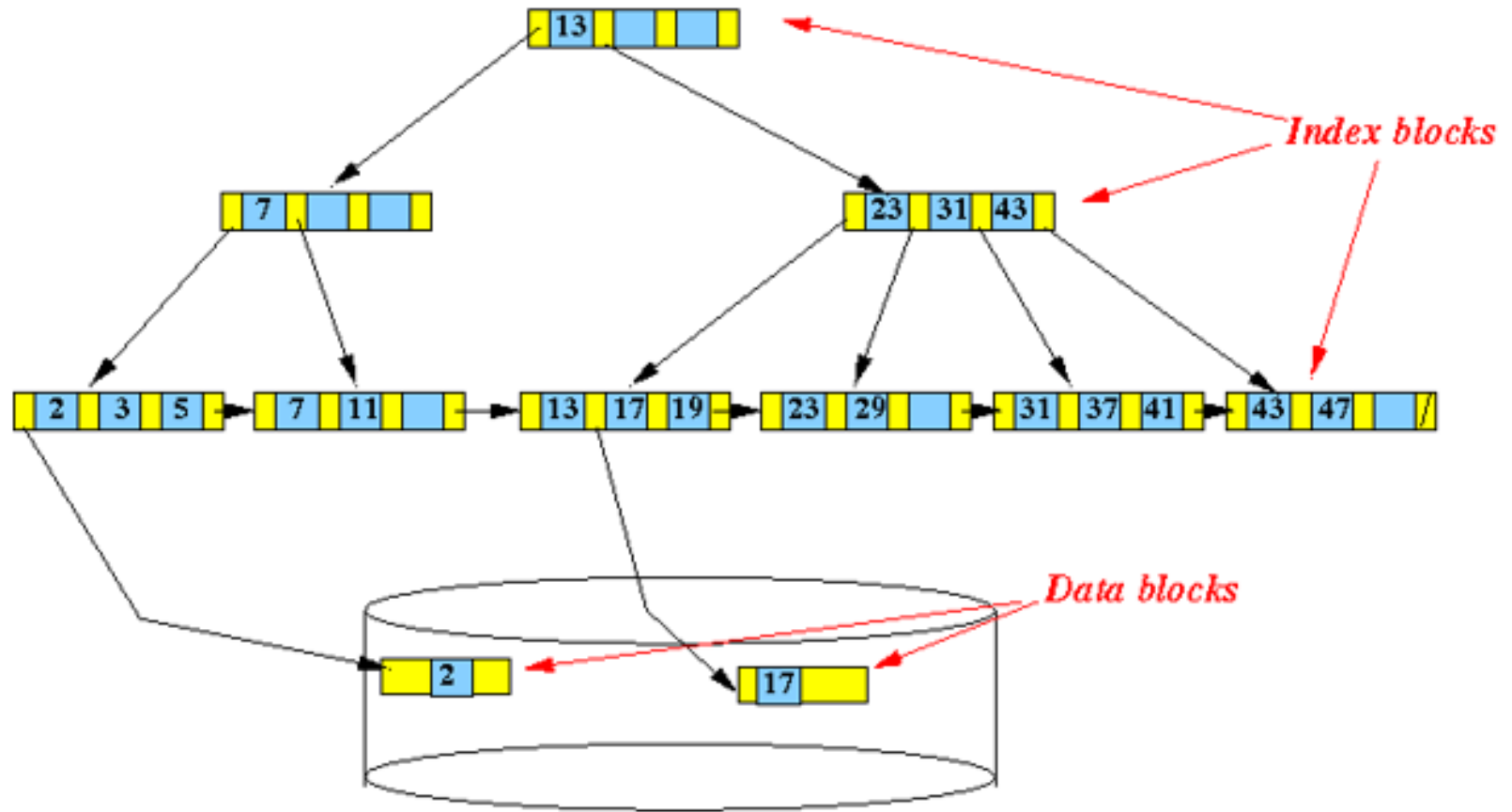
Cont'd...

- **S6 Using a secondary (B+-tree) index:**

- On an equality comparison, this search method can be used to retrieve a single record if the indexing field is a key or to retrieve multiple records if the indexing field is not a key.
- In addition, it can be used to retrieve records on conditions involving $>$, $>=$, $<$, or $<=$. (FOR RANGE QUERIES).
- In case of range queries, the B+ tree index leaf nodes contain the indexing field value in order so a sequence of them is used corresponding to the requested range of that field and provide record pointers to the qualifying records.

Cont'd...

- S6 Using a secondary (B+-tree) index:



Cont'd...

■ S7a—Using a bitmap index:

- If the selection condition involves a set of values for an attribute, the corresponding bitmaps for each value can be OR-ed to give the set of record ids that qualify.
- Is suitable for retrievals where an attribute must match an enumerated set of values.

- For example,

```
SELECT *
```

```
FROM EMPLOYEE
```

```
WHERE Dno IN (3,49).
```

- For the above query: record 1,2,6 and 4 is returned

Based on the given bitmap index.

Fields:	Dno	Name
1:	3	Ram
2:	3	Hari
3:	27	sita
4:	49	xyz
5:	27	gita
6:	3	abc

The bitmap index for the field Dno is
value 123456

3	110001	<---- bit vector
27	001010	
49	000100	

Explanation: The value 3 appears in the records: 1, 2, 6 So: bit #1, #2 and #6 are set

Cont'd...

- **S7b—Using a functional index:**

- Is suitable when the match is based on a function of one or more attributes on which a functional index exists

- For example,

```
SELECT First_name, Lname  
FROM Employee  
WHERE ((Salary*Commission_pct) + Salary ) > 15000;
```

- In above query, the selection condition involves the expression $((Salary * Commission_pct) + Salary)$. If there is a functional index defined as:

```
CREATE INDEX income_ix  
ON EMPLOYEE (Salary + (Salary*Commission_pct));
```

- then this index can be used to retrieve employee records that qualify.

Cont'd...

■ Search Methods for Conjunctive Selection:

- If a condition of a SELECT operation is a conjunctive condition that is, made up of several simple conditions connected with the AND logical connective such as:

$\sigma_{DNO=5 \text{ AND } SALARY>30000 \text{ AND } SEX=F}(\text{EMPLOYEE}).$

- The DBMS can use the following methods to implement the operation:
 - S8 Conjunctive selection
 - S9 Conjunctive selection using a composite index
 - S10 Conjunctive selection by intersection of record pointers:

Cont'd...

■ S8 Conjunctive selection:

- If an attribute involved in any single simple condition in the conjunctive condition has an access path, use that condition to retrieve the records and then check whether each retrieved record satisfies the remaining simple conditions in the conjunctive condition.

- For example,

```
SELECT *  
FROM EMPLOYEE  
WHERE Dno = 5 AND Salary > 50000;
```



Cont'd...

- **S9 Conjunctive selection using a composite index**

- If two or more attributes are involved in equality conditions in the conjunctive condition and a composite index exists on the combined field, we can use the index directly.
- For example, if an index has been created on the composite key (Essn, Pno) of the WORKS_ON file
- Then for $\sigma_{\text{ESSN}=123456789 \text{ AND } \text{PNO}=10}(\text{WORKS_ON})$ we can use the index directly.

Cont'd...

- **S10 Conjunctive selection by intersection of record pointers:**
 - This method requires secondary indexes are available on the fields involved in equality comparison conditions in the conjunctive condition and indexes include record pointers.
 - Each index can be used to **retrieve the record pointers** that satisfy the individual condition.
 - The **intersection** of these sets of record pointers gives the record pointers that satisfy the conjunctive condition, which are then used to **retrieve** those records directly.
 - If only some of the conditions have secondary indexes, each retrieved record is further tested to determine whether it satisfies the remaining conditions.

Cont'd...

- **S10 Conjunctive selection by intersection of record pointers:** Consider the query : `SELECT * FROM EMPLOYEE WHERE Dno = 5 AND Salary > 50000;`
- With index on Dno and Salary

Use index on Dno to retrieve record pointers of record having Dno =5.
e.g. {R1, R2, R3, R5}

Use index on Salary to retrieve record pointers of record having salary>50000.
e.g., {R2, R3, R6}

Find the Intersection of the record pointers e.g., Common records = {R2, R3}

Retrieve actual records by using these record pointers i.e., Fetch R2 and R3

Cont'd...

- **Search Methods for Disjunctive Selection:**

- Selection involving a disjunctive condition
 - For example, consider $\sigma_{Dno=5 \text{ OR } Salary > 30000 \text{ OR } Sex = 'F'}(EMPLOYEE)$
- Simple conditions are connected by the OR logical connective.
- With such a condition, the records satisfying the disjunctive condition are the union of the records satisfying the individual conditions.
 - If an access path exists on every simple condition in the disjunction then retrieve the records satisfying each condition and then apply the union operation to eliminate duplicates.
- Hence, if any one of the conditions does not have an access path, then the brute force, linear search approach is used.

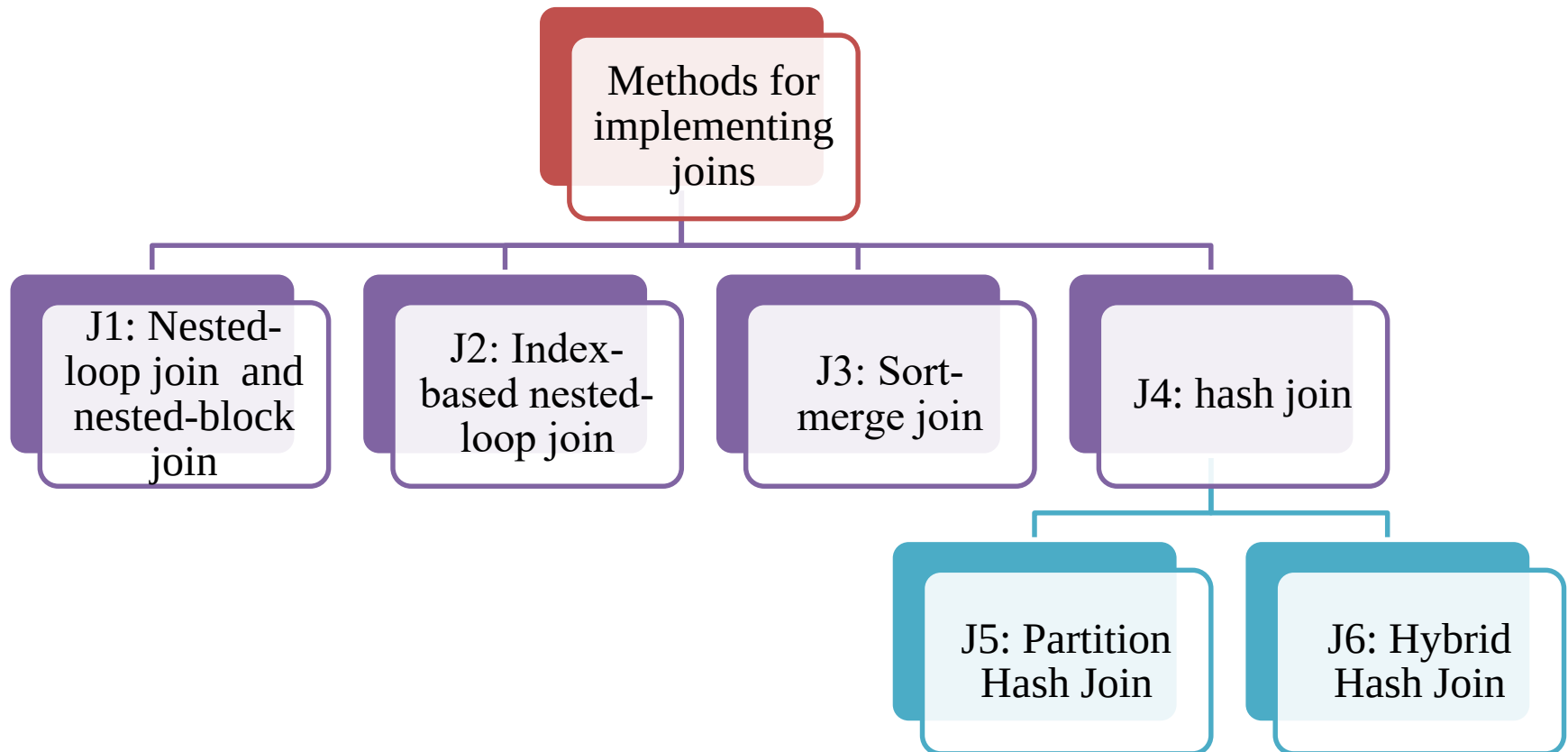
2.4. Implementing the JOIN Operation

■ JOIN operation

- One of the most time consuming operation in query processing
- EQUIJOIN is the most common one.
 - Two-way join: a join on two files
 - e.g. $R \bowtie_{A=B} S$
 - where A and B are the join attributes, which should be domain-compatible attributes of R and S, respectively.
 - Multi-way joins: joins involving more than two files.
 - e.g. $R \bowtie_{A=B} S \bowtie_{C=D} T$
- Examples,
 - $\text{EMPLOYEE} \bowtie_{\text{Dno}=\text{Dnumber}} \text{DEPARTMENT}$
 - $\text{DEPARTMENT} \bowtie_{\text{Mgr_ssn}=\text{Ssn}} \text{EMPLOYEE}$

Cont'd...

- **Methods for implementing joins:**



Cont'd...

- **J1 Nested-loop join (brute force):**

- For each record t in R (Relation in outer loop), retrieve every record s from S (Relation in inner loop) and test whether the two records satisfy the join condition $t[A] = s[B]$.

a. Nested-Loop Join Algorithm: To compute the join $R \bowtie_{A=B} S$

for each tuple t in R do begin

for each tuple s in S do begin

test pair (t, s) to see if they satisfy the join condition

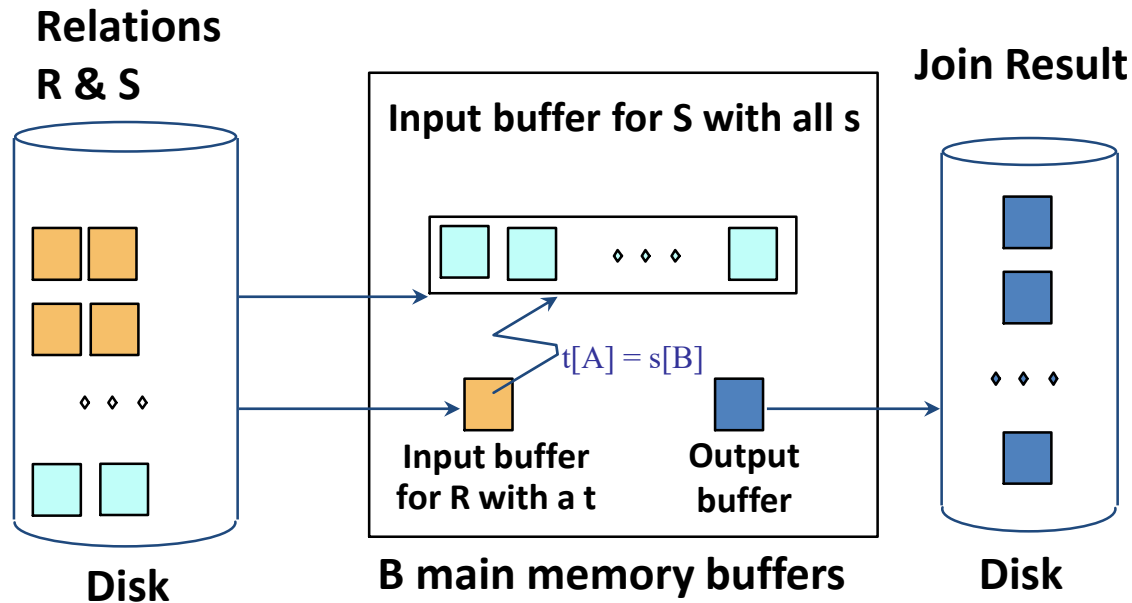
if they do, add $t \cdot s$ to the result.

end

end

Cont'd...

- a. Nested-Loop Join Algorithm:**



- R is called the outer relation and S the inner relation of the join.
- Requires no indices and can be used with any kind of join condition.
- Expensive since it examines every pair of tuples in the two relations.

Cont'd...

- **b. Block Nested-Loop Join Algorithm:** Variant of nested-loop join in which every block of inner relation is paired with every block of outer relation.

for each block B_r of R **do begin**

for each block B_s of S **do begin**

for each tuple t in B_r **do begin**

for each tuple s in B_s **do begin**

 Check if (t, s) satisfy the join condition

 if they do, add $t \bullet s$ to the result.

end

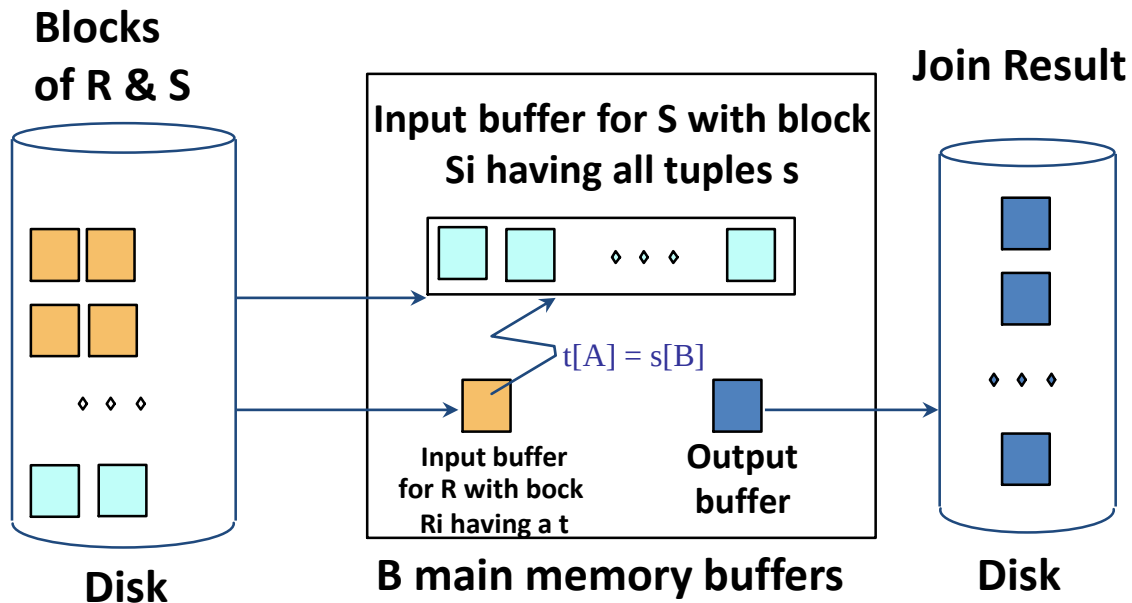
end

end

end

Cont'd...

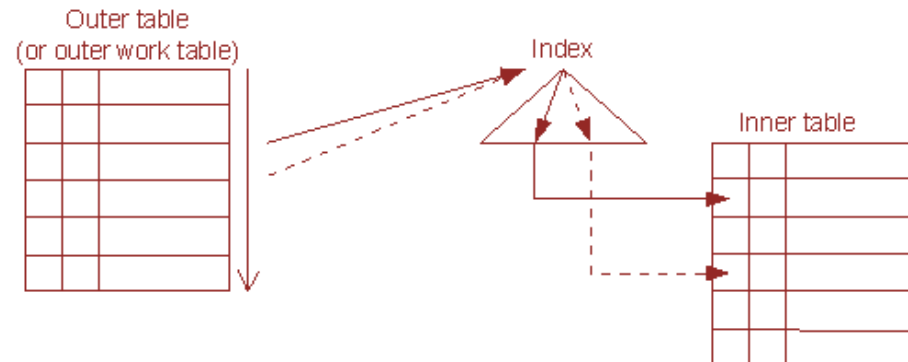
- b. Block Nested-Loop Join Algorithm:**



Cont'd...

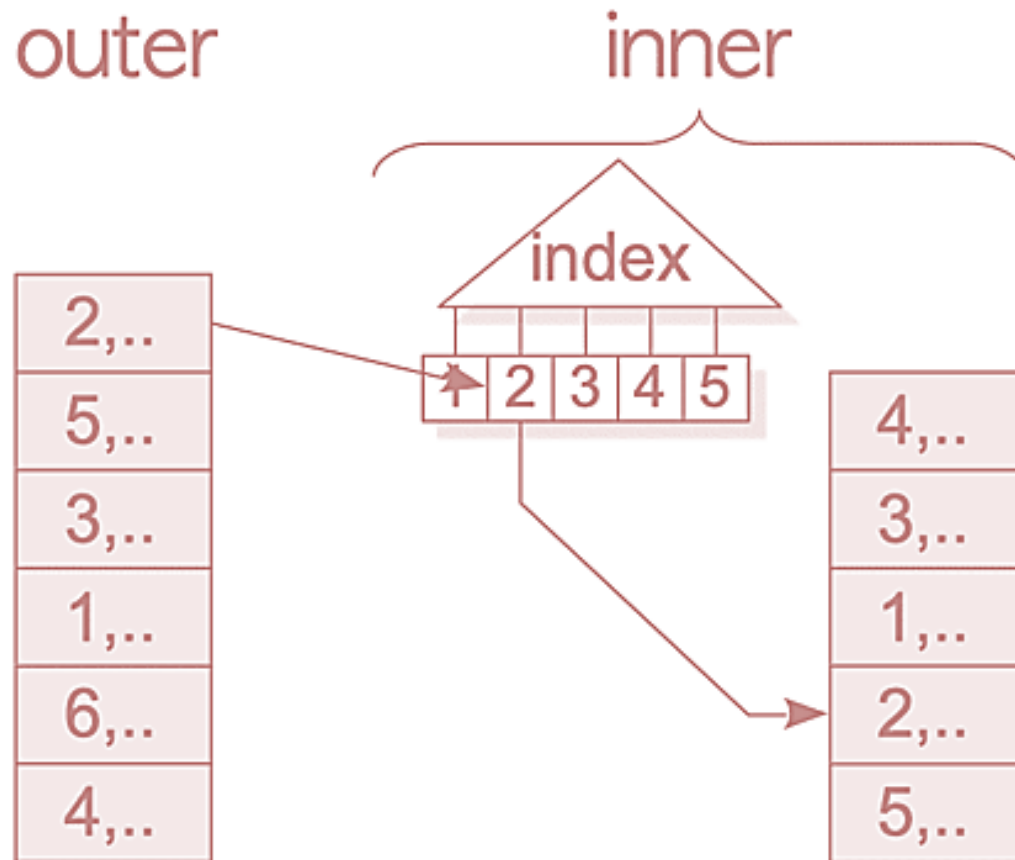
■ J2 Index-based nested-loop join:

- Using an access structure to retrieve the matching records.
- If an index exists for one of the two join attributes — say, B of S.
- Retrieve each record t in R , one at a time, and then use the access structure to retrieve directly all matching records s from S that satisfy $s[B] = t[A]$.
- If indices are available on join attributes of both R and S , use the relation with fewer tuples as the outer relation.



Cont'd...

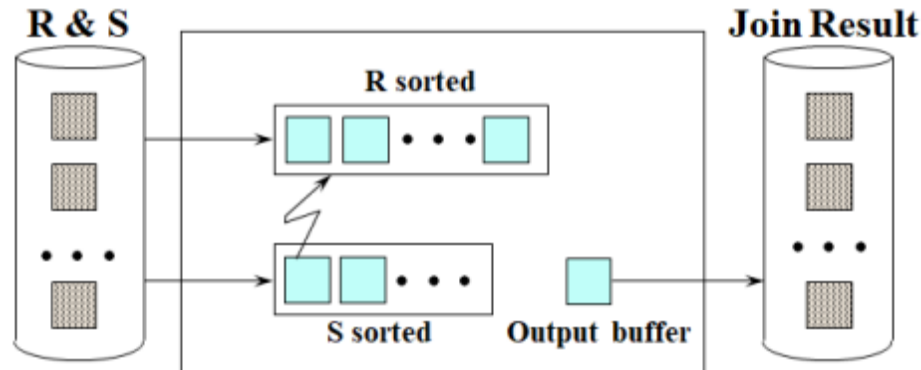
- **J2 Index-based nested-loop join Example:**



Cont'd...

■ J3 Sort-merge join:

- The records of R and S are *physically sorted* by value of the join attributes A and B, respectively.
- Both files are scanned in order of the join attributes, matching the records that have the same values for A and B.
- Matching records from R and S are combined to return the result of the join.



Cont'd...

```
(a) sort the tuples in  $R$  on attribute  $A$ ; (*assume  $R$  has  $n$  tuples (records)*)
    sort the tuples in  $S$  on attribute  $B$ ; (*assume  $S$  has  $m$  tuples (records)*)
    set  $i \leftarrow 1, j \leftarrow 1$ ;
    while ( $i \leq n$ ) and ( $j \leq m$ )
    do { if  $R(i)[A] > S(j)[B]$ 
        then set  $j \leftarrow j + 1$ 
      elseif  $R(i)[A] < S(j)[B]$ 
        then set  $i \leftarrow i + 1$ 
      else { (*  $R(i)[A] = S(j)[B]$ , so we output a matched tuple *)
            output the combined tuple  $\langle R(i), S(j) \rangle$  to  $T$ ;

            (* output other tuples that match  $R(i)$ , if any *)
            set  $l \leftarrow j + 1$ ;
            while ( $l \leq m$ ) and ( $R(i)[A] = S(l)[B]$ )
            do { output the combined tuple  $\langle R(i), S(l) \rangle$  to  $T$ ;
                set  $l \leftarrow l + 1$ 
            }

            (* output other tuples that match  $S(j)$ , if any *)
            set  $k \leftarrow i + 1$ ;
            while ( $k \leq n$ ) and ( $R(k)[A] = S(j)[B]$ )
            do { output the combined tuple  $\langle R(k), S(j) \rangle$  to  $T$ ;
                set  $k \leftarrow k + 1$ 
            }
            set  $i \leftarrow k, j \leftarrow l$ 
          }
    }
```

Figure: (a) Implementing JOIN, by using sort-merge, where R has n tuples and S has m tuples

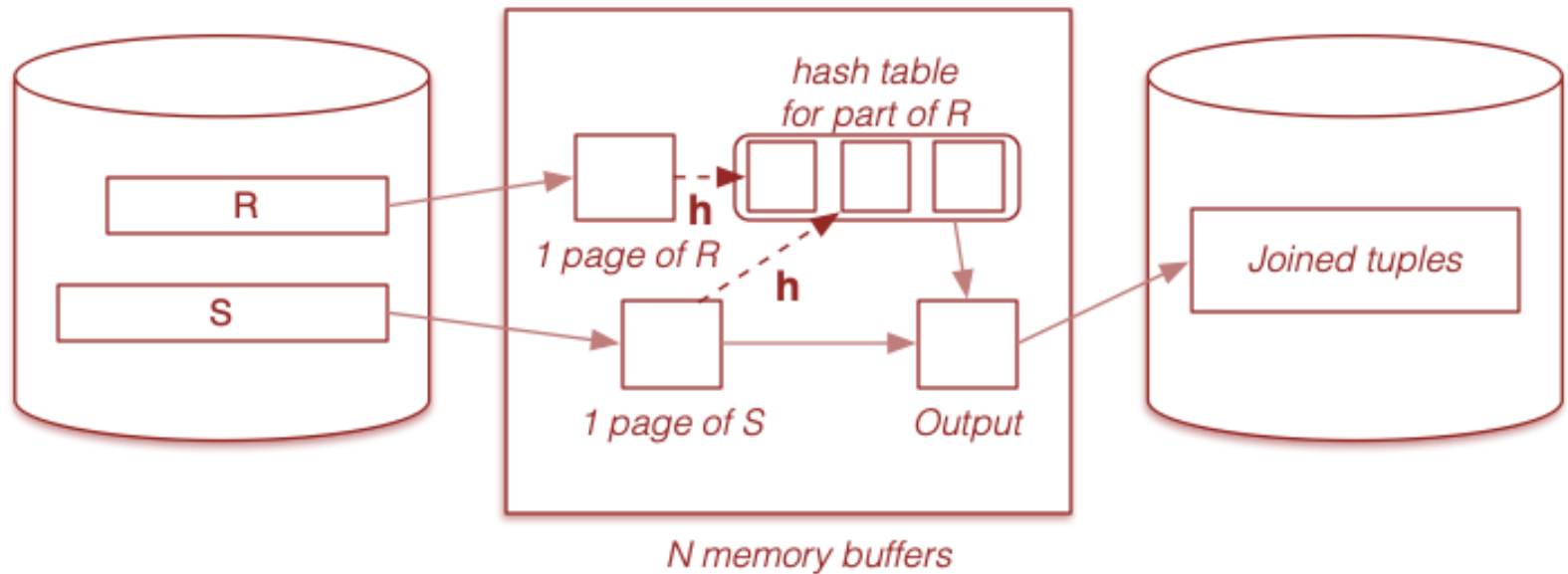
Cont'd...

■ J4 Hash-join:

- The records of files R and S are both hashed to the *same hash file*, using the *same hashing function* on the join attributes A of R and B of S as hash keys.
- A single pass through the file with fewer records (say, R) hashes its records to the hash file buckets.
- A single pass through the other file (S) then hashes each of its records to the appropriate bucket, where the record is combined with all matching records from R.

Cont'd...

■ J4 Hash-join Example:



Cont'd...

■ J5 Partition hash join

■ Partitioning phase:

- Each file (R and S) is first partitioned into M partitions using a partitioning hash function on the join attributes:
 - $R_1, R_2, R_3, \dots, R_m$ and $S_1, S_2, S_3, \dots, S_m$
- Minimum number of in-memory buffers needed for the partitioning phase: $M+1$.
- A disk sub-file is created per partition to store the tuples for that partition.

■ Joining or probing phase:

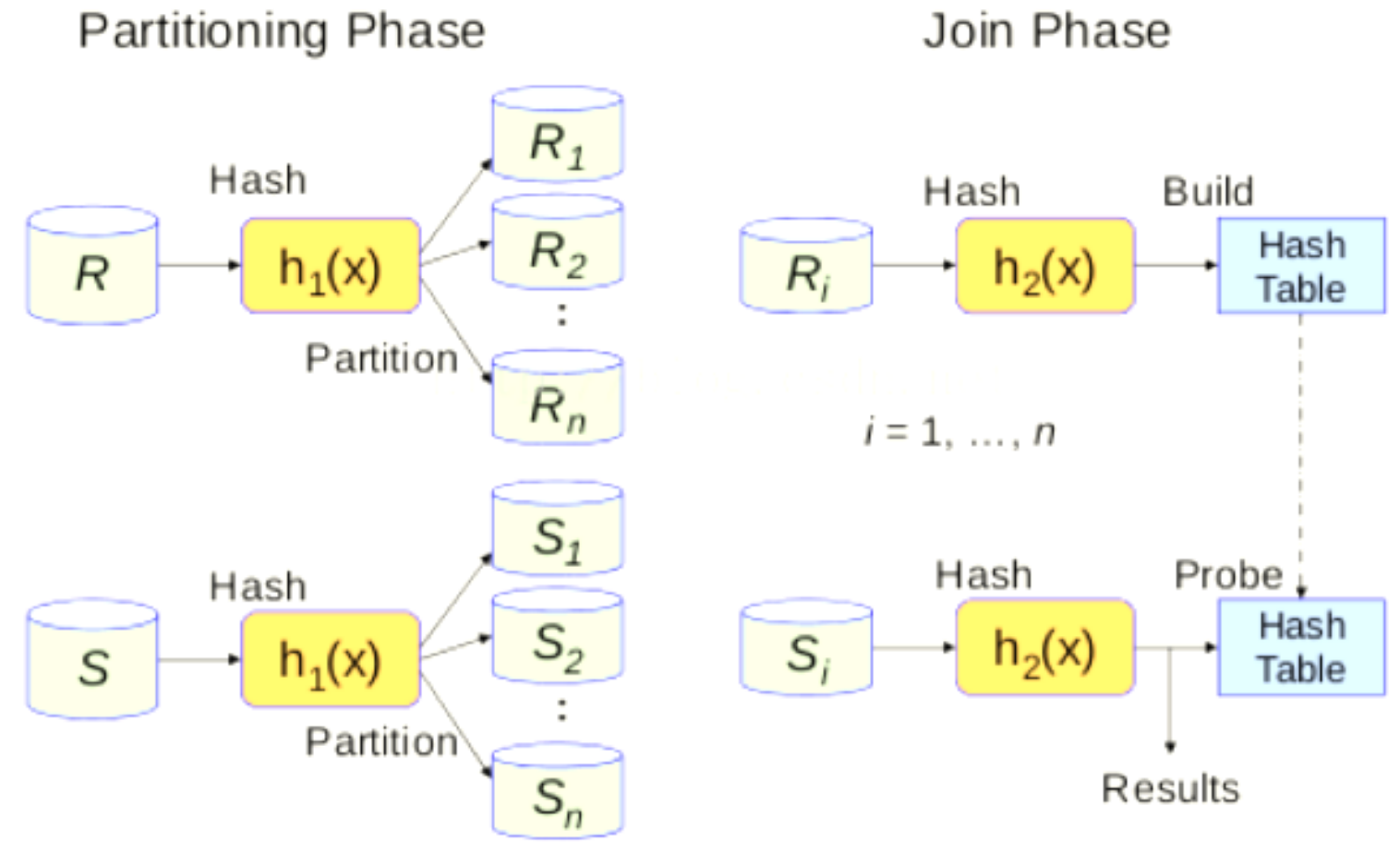
- Involves M iterations, one per partitioned file.
- Iteration i involves joining partitions R_i and S_i .

Cont'd...

- **Partitioned Hash Join Procedure:** Assume R_i is smaller than S_i .
 1. Copy records from R_i into memory buffers.
 2. Read all blocks from S_i , one at a time and each record from S_i is used to *probe (that is, search)* for a matching record(s) from partition R_i .
 3. Write matching record from R_i after joining to the record from S_i into the result file.

Cont'd...

■ Partitioned Hash Join Example:



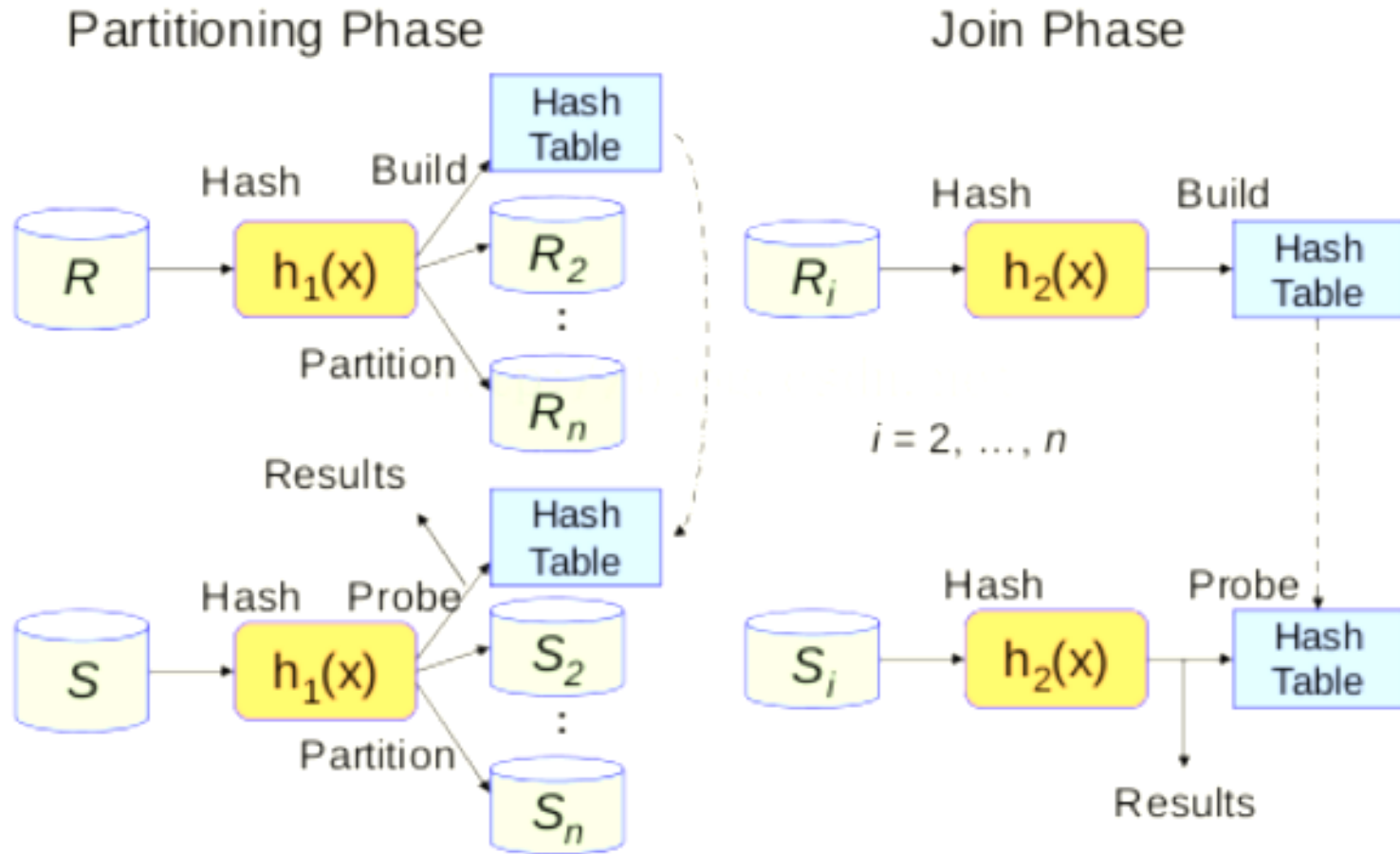
Cont'd...

■ J6 Hybrid hash join:

- Same as partitioned hash join except:
 - Joining phase of one of the partitions is included during the partitioning phase.
- Partitioning phase:
 - Allocate buffers for smaller relation- one block for each of the M-1 partitions, remaining blocks to partition 1.
 - Repeat for the larger relation in the pass through S.
- Joining phase:
 - M-1 iterations are needed for the partitions R2 , R3 , R4 ,Rm and S2 , S3 , S4 ,Sm. R1 and S1 are joined during the partitioning of S1, and results of joining R1 and S1 are already written to the disk by the end of partitioning phase.

Cont'd...

■ J6 Hybrid hash join Example :



Cont'd...

- **Factors affecting JOIN performance:**
 - Available buffer space
 - Join selection factor
 - Choice of inner VS outer relation

2.5 Algorithms for PROJECT and Set Operations

■ 2.5.1. Algorithms for PROJECT Operations - PROJECT operation:

Algorithm for PROJECT operations: $\pi_{\langle \text{attribute list} \rangle}(R)$

1. If $\langle \text{attribute list} \rangle$ has a key of relation R , extract all tuples from R with only the values for the attributes in $\langle \text{attribute list} \rangle$.
 2. If $\langle \text{attribute list} \rangle$ does NOT include a key of relation R , duplicated tuples must be removed from the results.
- Methods to remove duplicate tuples: Sorting and Hashing

Cont'd...

- **2.5.1. Algorithms for PROJECT Operations: PROJECT operation**

(b) create a tuple $t[\langle \text{attribute list} \rangle]$ in T' for each tuple t in R ;
(* T' contains the projection results *before* duplicate elimination *)
if $\langle \text{attribute list} \rangle$ includes a key of R
then $T \leftarrow T'$
else { sort the tuples in T' ;
set $i \leftarrow 1, j \leftarrow 2$;
while $i \leq n$
do { output the tuple $T'[i]$ to T ;
while $T'[i] = T'[j]$ and $j \leq n$ do $j \leftarrow j + 1$; (* eliminate duplicates *)
 $i \leftarrow j; j \leftarrow i + 1$
}
}
(* T contains the projection result after duplicate elimination*)

Figure : Implementing the PROJECT operation

Cont'd...

■ 2.5.2. Algorithms for SET Operations –

■ Set operations includes the:

- UNION,

- INTERSECTION,

- SET DIFFERENCE and

- CARTESIAN PRODUCT

■ Set operations sometimes expensive to implement: Sort-merge technique and Hashing

Cont'd...

■ 2.5.2. Algorithms for SET Operations - UNION:

- Sort the two relations on the same attributes.
- Scan and merge both sorted files concurrently, whenever the same tuple exists in both relations, only one is kept in the merged results.

```
(c) sort the tuples in  $R$  and  $S$  using the same unique sort attributes;  
    set  $i \leftarrow 1, j \leftarrow 1$ ;  
    while  $(i \leq n)$  and  $(j \leq m)$   
    do { if  $R(i) > S(j)$   
        then { output  $S(j)$  to  $T$ ;  
              set  $j \leftarrow j + 1$   
        }  
        elseif  $R(i) < S(j)$   
        then { output  $R(i)$  to  $T$ ;  
              set  $i \leftarrow i + 1$   
        }  
        else set  $j \leftarrow j + 1$  (*  $R(i)=S(j)$ , so we skip one of the duplicate tuples *)  
    }  
    if  $(i \leq n)$  then add tuples  $R(i)$  to  $R(n)$  to  $T$ ;  
    if  $(j \leq m)$  then add tuples  $S(j)$  to  $S(m)$  to  $T$ ;
```

Figure: Implementing the UNION operation

Cont'd...

■ 2.5.2. Algorithms for SET Operations - INTERSECTION

- Sort the two relations on the same attributes.
- Scan and merge both sorted files concurrently, keep in the merged results only those tuples that appear in both relations.

```
(d) sort the tuples in  $R$  and  $S$  using the same unique sort attributes;  
    set  $i \leftarrow 1, j \leftarrow 1$ ;  
    while  $(i \leq n)$  and  $(j \leq m)$   
    do {   if  $R(i) > S(j)$   
           then set  $j \leftarrow j + 1$   
         elseif  $R(i) < S(j)$   
           then set  $i \leftarrow i + 1$   
         else {   output  $R(j)$  to  $T$ ;           (*  $R(i) = S(j)$ , so we output the tuple *)  
                 set  $i \leftarrow i + 1, j \leftarrow j + 1$   
         }  
    }
```

Figure: Implementing the INTERSECTION operation

Cont'd...

■ 2.5.2. Algorithms for SET Operations - SET DIFFERENCE R-S

- Keep in the merged results only those tuples that appear in relation R but not in relation S.

(e) sort the tuples in R and S using the same unique sort attributes;

set $i \leftarrow 1, j \leftarrow 1$;

while $(i \leq n)$ and $(j \leq m)$

do { if $R(i) > S(j)$

then set $j \leftarrow j + 1$

elseif $R(i) < S(j)$

then { output $R(i)$ to T ;

(* $R(i)$ has no matching $S(j)$, so output $R(i)$ *)

set $i \leftarrow i + 1$

}

else set $i \leftarrow i + 1, j \leftarrow j + 1$

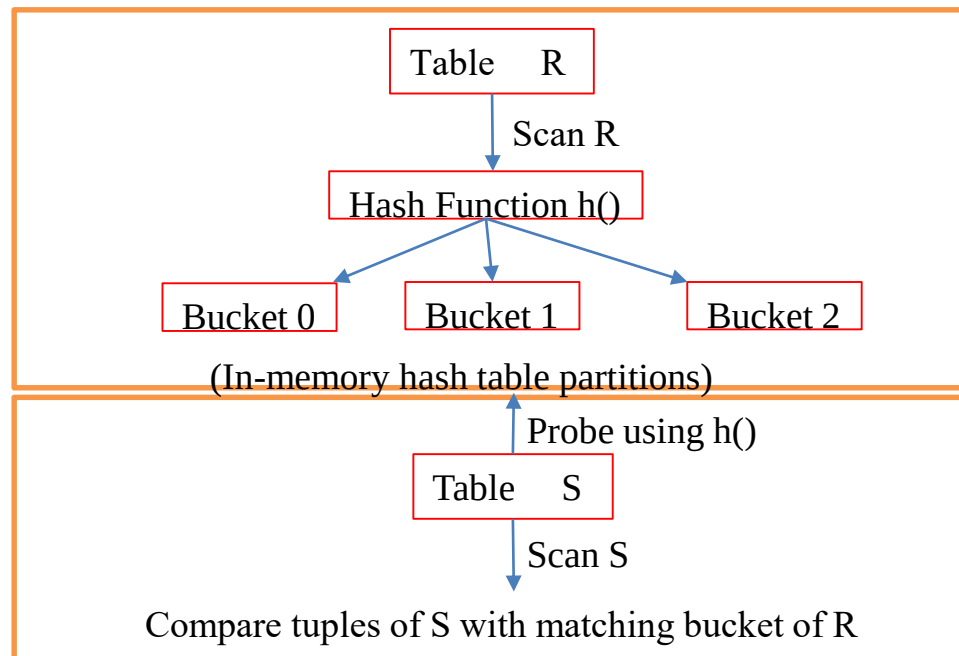
}

if $(i \leq n)$ then add tuples $R(i)$ to $R(n)$ to T ;

Figure: Implementing the SET DIFFERENCE operation

Cont'd...

- **Algorithms for Set Operation using Hashing:** Hashing Can also be used to implement set operations ($\cup$, $\cap$, and $-$) as:
 - One table is first scanned and then partitioned into an in-memory hash table with buckets, and
 - The records in the other table are then scanned one at a time and used to probe the appropriate partition.



Cont'd...

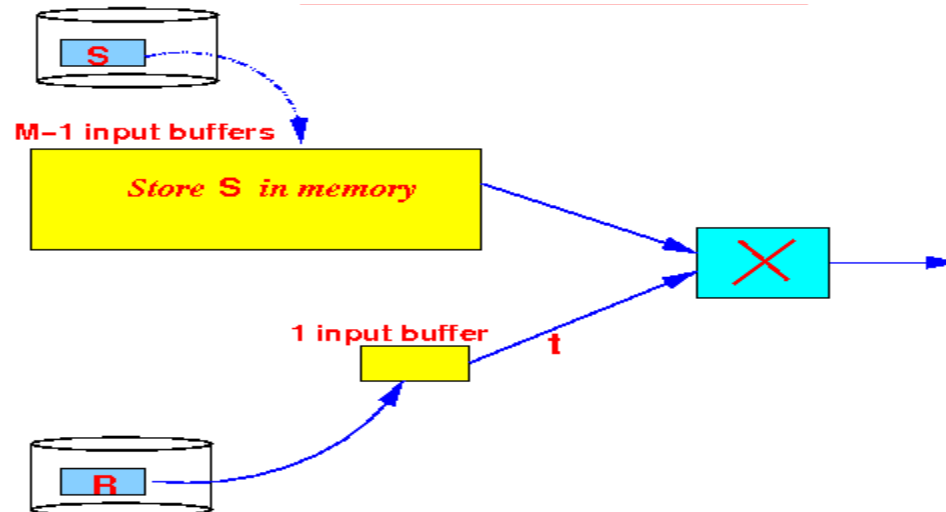
- **Algorithms for Set Operation using Hashing:**

- To implement $\mathbf{R \cup S}$, first hash (partition) the records of R; then, hash (probe) the records of S, but do not insert duplicate records in the buckets.
- To implement $\mathbf{R \cap S}$, first partition the records of R to the hash file. Then, while hashing each record of S, probe to check if an identical record from R is found in the bucket, and if so add the record to the result file.
- To implement $\mathbf{R - S}$, first hash the records of R to the hash file buckets. While hashing (probing) each record of S, if an identical record is found in the bucket, remove that record from the bucket.

Cont'd...

■ 2.5.2. Algorithms for SET Operations – CARTESIAN PRODUCT

- CARTESIAN PRODUCT of relations R and S include all possible combinations of records from R and S.



- The attribute of the result include all attributes of R and S.
- **Cost analysis** of CARTESIAN PRODUCT
 - If R has n records and j attributes and S has m records and k attributes, the result relation will have $n*m$ records and $j+k$ attributes.
- CARTESIAN PRODUCT operation is **very expensive** and should be avoided if possible.

Cont'd...

- **2.5.2. Algorithms for SET Operations - Use of anti-join for SET DIFFERENCE (EXCEPT or MINUS in SQL)**

- The MINUS operator in SQL is transformed into an anti-join as follows.

- Example: Find which departments have no employees

```
SELECT Dnumber FROM DEPARTMENT
```

```
MINUS
```

```
SELECT Dno FROM EMPLOYEE;
```

- Can be converted using anti-join operation into

```
SELECT DISTINCT DEPARTMENT.Dnumber
```

```
FROM DEPARTMENT, EMPLOYEE
```

```
WHERE DEPARTMENT.Dnumber A = EMPLOYEE.Dno;
```

Cont'd...

- And, the query

```
SELECT Dno
```

```
from EMPLOYEE
```

```
INTERSECT ALL
```

```
SELECT Dum from PROJECT;
```

- Can be converted using semi-join operation into

```
SELECT DISTINCT EMPLOYEE.Dno
```

```
FROM DEPARTMENT, EMPLOYEE
```

```
WHERE EMPLOYEE.Dno S = DEPARTMENT.Dnumber;
```

2.6. Implementing Aggregate Operations and Different Types of JOINS

- **2.6.1 Implementing Aggregate Operations:** Aggregate operators includes the following:
 - MIN,
 - MAX,
 - COUNT,
 - AVERAGE,
 - SUM
- Can be computed by a table scan or using an appropriate index, if available

Cont'd...

■ 2.6.1 Implementing Aggregate Operations: MAX and MIN

■ Example,

Select MAX (Salary)

From EMPLOYEE

- If an index exist on an attribute (e.g., ascending B+ tree index on Salary exists):
 - Optimizer can use the Salary index to search for the largest Salary value i.e., **MAX**
 - Follow the rightmost pointer in each index node from the root to the **rightmost** leaf
 - The **MIN** function uses leftmost pointer in the index from the root to **leftmost** leaf.
- Otherwise use the full table scan.

Cont'd...

- **2.6.1 Implementing Aggregate Operations: AVERAGE or SUM**
 - Index can be used if it is a dense index
 - Computation applied to the values in the index
 - Non-dense index can be used if actual number of records associated with each index value is stored in each index entry
- **COUNT:** Number of values can be computed from the index or from the catalog.

Cont'd...

■ 2.6.1 Implementing Aggregate Operations: **GROUP BY**

- When a **GROUP BY** clause is used in a query, the aggregate operator must be applied separately to each group of tuples as partitioned (either using sorting or hashing) by the grouping attribute.
- If a clustering index exists on the grouping attribute(s), then the records are already partitioned (grouped) into the appropriate subsets. In this case, it is only necessary to apply the aggregate operator to each group.

Cont'd...

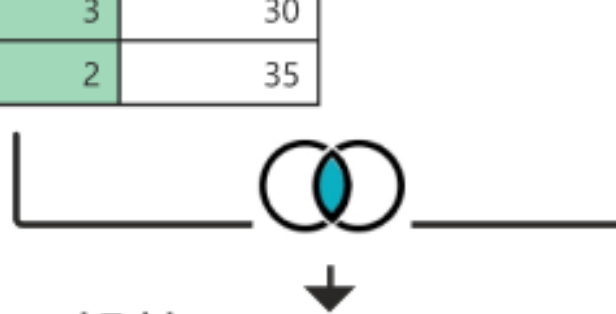
- **2.6.2. Implementing Different Types of JOINS:**
 - Standard JOIN called INNER JOIN in SQL.

Left Table

Date	CountryID	Units
1/1/2024	1	40
1/2/2024	1	25
1/3/2024	3	30
1/4/2024	2	35

Right Table

ID	Country
3	Panama
4	Spain



Merged Table

Date	CountryID	Units	Country
1/3/2024	3	30	Panama

Cont'd...

■ 2.6.2. Implementing Different Types of JOINS:

- Variations of joins:
 - Outer join:
 - Left,
 - Right, and
 - Full
 - Semi-Join
 - Anti-Join
 - Non-Equi-Join
- These Joins can be implemented by modifying the join-algorithms such as Nested Loop, Sort-Merge or hash- join algorithms.

Cont'd...

- **2.6.2. Implementing Different Types of JOINS: Outer Joins**
 - **Outer joins** can be implemented by modifying any one join algorithm as:
 - For **left outer join**, use the left relation as outer relation and construct result from every tuple in the left relation.
 - If there are matching tuples in the other relation, the joined tuples are produced and saved in the result.
 - However, if no matching tuple is found, the tuple is still included in the result but is padded with NULL value(s).

Cont'd...

■ 2.6.2. Implementing Different Types of JOINS: Left Outer Joins

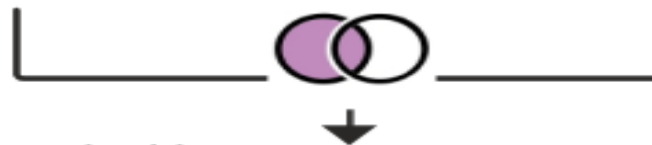
- For example,

Left Table

Date	CountryID	Units
1/1/2024	1	40
1/2/2024	1	25
1/3/2024	3	30
1/4/2024	4	35

Right Table

ID	Country
1	USA
2	Canada
3	Panama



Merged Table

Date	CountryID	Units	Country
1/1/2024	1	40	USA
1/2/2024	1	25	USA
1/3/2024	3	30	Panama
1/4/2024	4	35	<i>null</i>

Cont'd...

■ 2.6.2. Implementing Different Types of JOINS: Right Outer Joins

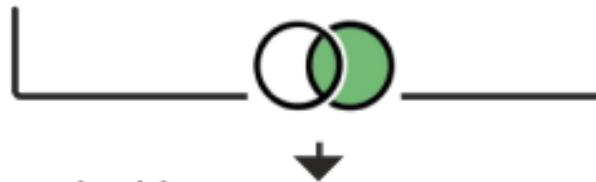
- The **right outer join** can be converted to a left outer join by switching the operands (i.e., use any one join algorithm by making right table as outer table and left table as inner table).

Left Table

Date	CountryID	Units
1/1/2020	1	40
1/2/2020	1	25
1/3/2020	3	30
1/4/2020	4	35

Right Table

ID	Country
3	Panama



Merged Table

Date	CountryID	Units	Country
1/3/2020	3	30	Panama

Cont'd...

■ 2.6.2. Implementing Different Types of JOINS: Full Outer Joins

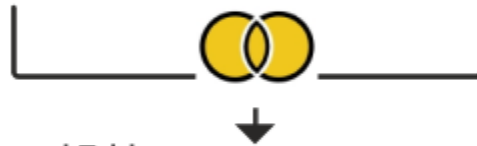
- Full outer join requires computing the result of inner join and then padding to the result extra tuples arising from unmatched tuples from both the left and right operand relations.

Left Table

Date	CountryID	Units
1/1/2024	1	40
1/2/2024	1	25
1/3/2024	3	30
1/4/2024	2	35

Right Table

ID	Country
1	USA
2	Canada
3	Panama
4	Spain



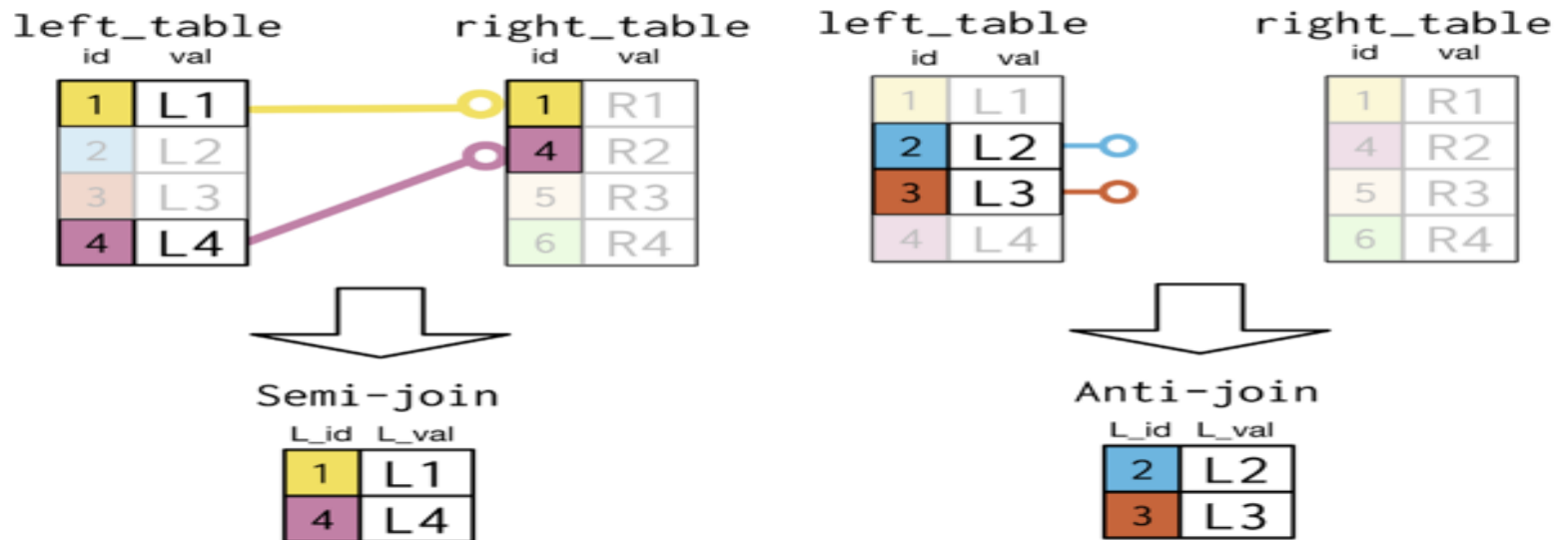
Merged Table

Date	CountryID	Units	Country
1/1/2024	1	40	USA
1/2/2024	1	25	USA
1/4/2024	2	35	Canada
1/3/2024	3	30	Panama
null	null	null	Spain

Cont'd...

■ 2.6.2. Implementing Different Types of JOINS: Semi-Join and Anti-Join

- In **semi-join**, the search stops as soon as the first match is found and the tuple from outer relation is selected;
- in **anti-join**, search stops as soon as the first match is found and the tuple from outer relation is rejected.



- Both these types of joins can also be implemented as an extension of the join algorithms.

Cont'd...

■ 2.6.2. Implementing Different Types of JOINS: Non-Equi-Join:

- Join operation may also be performed when the join condition is one of inequality, such as $<$, $>$, $\geq$, $\leq$, and $\neq$.
- All of the join methods discussed are again applicable here with the exception that hash-based algorithms cannot be used.
- For example,

for each tuple $t1$ in Relation R do

for each tuple $t2$ in Relation S do

if $inequality_condition(t1, t2)$ is TRUE then

output $(t1, t2)$

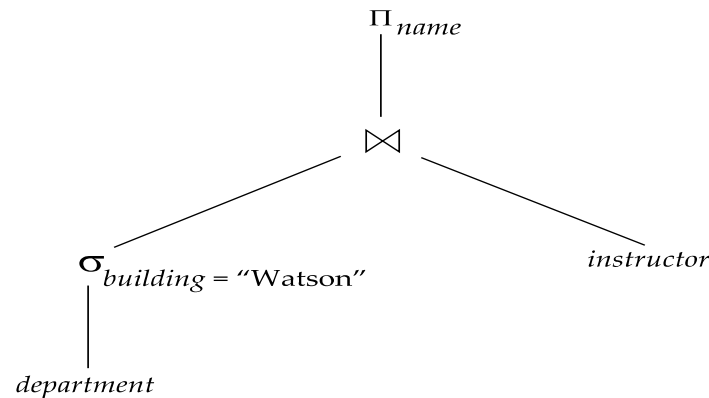
2.7. Combining Operations Using Pipelining

- SQL query translated into relational algebra expression is a Sequence of relational operations.
- Alternatives for evaluating an entire algebra expression:
 - **Materialization or Materialized evaluation:** Creating, storing, and passing temporary results. i.e., generate results of an expression whose inputs are relations or are already computed, materialize (store) it on disk. Repeat.
 - However, the General query goal is to minimize the number of temporary files
 - **Pipelining or Pipelined evaluation:** Combines several operations into one in order to avoid writing temporary files. i.e., pass on tuples to parent operations even as an operation is being executed. Pipelined evaluation offers the following benefits:
 - Avoiding cost and time delay associated with writing intermediate results to disk
 - Being able to start generating results as quickly as possible

Cont'd...

■ Materialized evaluation:

- Evaluate one operation at a time, starting at the lowest-level. Use intermediate results materialized into temporary relations to evaluate next-level operations.
- E.g., in figure below, compute and store $\sigma_{\text{building}=\text{"Watson"}}(\text{department})$.
- then compute the store its join with instructor, and finally compute the projection on name.



- Materialized evaluation is always applicable but Cost of writing results to disk and reading them back can be quite high.

Cont'd...

■ Pipelined evaluation:

- Evaluate several operations simultaneously, passing the results of one operation on to the next.
- E.g., in previous expression tree, don't store result of $\sigma_{\text{building}=\text{"Watson"}}$ (department).
- instead, pass tuples directly to the join.. Similarly, don't store result of join, pass tuples directly to projection.
- Much cheaper than materialization: no need to store a temporary relation to disk.
- Pipelining may not always be possible – e.g., sort, hash-join.
- For pipelining to be effective, use evaluation algorithms that generate output tuples even as tuples are received for inputs to the operation.
- Pipelines can be executed in two ways: **demand driven** and **producer driven**

Cont'd...

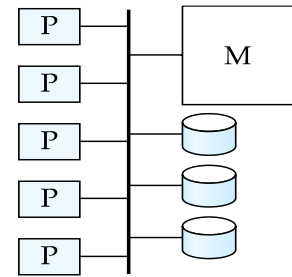
- **In demand driven or lazy evaluation**
 - system repeatedly requests next tuple from top level operation
 - Each operation requests next tuple from children operations as required, in order to output its next tuple
 - In between calls, operation has to maintain “state” so it knows what to return next
- **In producer-driven or eager pipelining**
 - Operators produce tuples eagerly and pass them up to their parents
 - Buffer maintained between operators, child puts tuples in buffer, parent removes tuples from buffer
 - if buffer is full, child waits till there is space in the buffer, and then generates more tuples
 - System schedules operations that have space in output buffer and can process more input tuples
- Alternative name: **pull** and **push** models of pipelining

Cont'd...

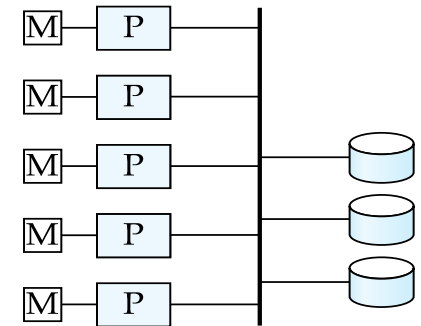
- **Implementation of demand-driven pipelining:** Each operation is implemented as an **iterator** implementing the following operations
 - **open()**
 - E.g., file scan: initialize file scan
 - state: pointer to beginning of file
 - E.g., merge join: sort relations;
 - state: pointers to beginning of sorted relations
 - **next()**
 - E.g., for file scan: Output next tuple, and advance and store file pointer
 - E.g., for merge join: continue with merge from earlier state till next output tuple is found. Save pointers as iterator state.
 - **close()**

2.8. Parallel Algorithms for Query Processing

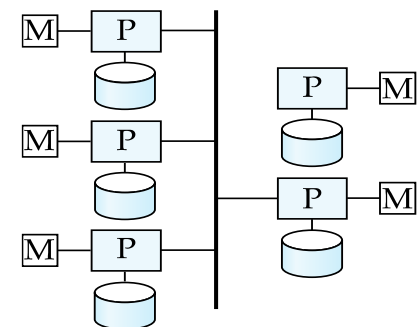
- Parallel database systems consist of multiple processors, memory and disks connected by a fast interconnection network.
- Goal is to handle workloads beyond what a single computer system can handle
- Depending on how processors, memory, and disks are organized three main parallel database architectures:
 - **Shared-memory architecture:** Multiple processors can access common main memory region
 - **Shared-disk architecture:** Every processor has its own memory but Machines have access to all disks
 - **Shared-nothing architecture:** Each processor has own memory and disk storage
 - Most commonly used in parallel database systems



(a) shared memory



(b) shared disk



(c) shared nothing

Cont'd...

- What ever be the parallel database architecture the implicit goals of parallel processing includes the following two by allocating more processors and disks proportional to the size of data:
 - **Linear speed-up**
 - Linear reduction in time taken for operations
 - **Linear scale-up**
 - Constant sustained performance.
- Parallelism in query processing can be achieved at the following three levels:
 - Individual operator parallelism,
 - Intraquery parallelism, and
 - Interquery parallelism

Cont'd...

■ 1) Operator-level parallelism:

- In the operations that can be implemented with parallel algorithms, one of the main strategies is to partition data across disks.
- Horizontal partitioning of a relation corresponds to distributing the tuples across disks based on following partitioning method.
 - Round-robin partitioning: Given n disks, assigning the i th tuple to disk $i \bmod n$ is called round-robin partitioning.
 - Range partitioning: tuples are equally distributed by dividing the range of values of some attribute.
 - Hash partitioning: tuple i is assigned to the disk $h(i)$, where h is the hashing function.

Cont'd...

- **1) Operator-level parallelism: Sorting**
 - If data has been range-partitioned on an attribute:
 - Each partition can be sorted separately in parallel
 - Results concatenated
 - If the relation has been partitioned using another scheme, the following approaches are possible:
 - Repartition the relation by using range partitioning on the same attribute that is the target for sorting; then sort each partition individually followed by concatenation,
 - Use parallel external sort-merge
 - Reduces sorting time: Achieves near n-fold speed-up

Cont'd...

■ 1) Operator-level parallelism: Selection

- For a selection based on some condition, if the condition is an equality condition, $\langle A = v \rangle$ and the same attribute A has been used for range partitioning, the selection can be performed on only that partition to which the value v belongs.
- In other cases, the selection would be performed in parallel on all the processors and the results merged.
- If the selection condition is $v1 \leq A \leq v2$ and attribute A is used for range partitioning, then the range of values $(v1, v2)$ must overlap a certain number of partitions.
- The selection operation needs to be performed only in those processors in parallel.

Cont'd...

- **1) Operator-level parallelism: Projection**
 - A) Projection without duplicate elimination
 - Perform operation in parallel as data is read
 - B) Duplicate elimination
 - Sort tuples and discard duplicates

Cont'd...

- **1) Operator-level parallelism: Parallel join**
 - Parallel joins divide the join into n smaller joins
 - Perform smaller joins in parallel on n processors
 - Take a union of the result
 - Parallel join techniques
 - Equality-based partitioned join
 - Inequality join with partitioning and replication
 - Parallel partitioned hash join

Cont'd...

- **1) Operator-level parallelism: Parallel join techniques:** a) Equality-based partitioned join
 - Relations R and S are partitioned into n parts
 - Corresponding partitions (r_i and s_i) are placed on the same processor P_i
 - Each processor performs a local join by using any one of Nested loop join, Sort-merge join or Hash join.
 - Assumptions or Prerequisites:
 - Join must be Equi-join or natural join
 - Both relations must be partitioned on the same join attribute and Using Same range partitioning, or Same hash function
 - Partitions must be non-overlapping
 - Ideal case: Equal-sized partitions result in balanced load
 - Problem: Data skewness results uneven workload reduces efficiency

Cont'd...

- **1) Operator-level parallelism: Parallel join techniques:** b) Inequality join with partitioning and replication:
 - Used when join condition involves $<, \leq, >, \geq, \neq$.
 - Asymmetric
 - Partition large relation R
 - Replicate smaller relation S to all processors
 - Each processor computes $r_i \bowtie S$
 - Efficient when S is small
 - Symmetric
 - Partition both relations R into n partitions and S into m partitions.
 - Execute using $m \times n$ processors so that each processor performs $r_i \bowtie s_j$
 - r_i replicated across row processors and s_j replicated across column processors
 - High replication cost and Expensive compared to equi-join

Cont'd...

- **1) Operator-level parallelism: Parallel join techniques:** c) Parallel partitioned hash join:
 - Used for large relations involves the following steps:
 - Distribution Phase: Apply hash function h_1 on join attribute and Distribute tuples of R and S to processors.
 - Partitioning Phase: At each processor partition S using another hash function h_2 into buckets.
 - Redistribution of R: Hash R using h_1 and send to processors and Partition R using same h_2
 - Local Join Phase: Perform hash join on corresponding buckets, Each processor produces partial result
 - Combine (union) all local results
 - Hash-based methods are most efficient for large data

Cont'd...

- **1) Operator-level parallelism: Aggregation**
 - Achieved by partitioning on the grouping attribute and then computing the aggregate function locally at each processor

Cont'd...

- **1) Operator-level parallelism:** Set operations
 - If argument relations are partitioned using the same hash function, they can be done in parallel on each processor

Cont'd...

- **2) Intraquery parallelism:** To achieve parallel execution of a query two main approaches:
 - Use parallel algorithm for each operation, with appropriate partitioning of the data input to that operation
 - Execute independent operations in parallel on separate processor.

Cont'd...

■ 3) Interquery parallelism

- Refers to execution of multiple queries in parallel on different processor.
- Goal is to scale up i.e., to increase the overall rate at which queries can be processed by increasing the number of processors
- For interquery parallelism requires:
 - Activities of locking, logging, and so on among processors must be coordinated, and simultaneous conflicting updates of the same data by multiple processors must be avoided.
 - There must be cache coherency, which guarantees that the processor updating a page has the latest version of that page in the buffer.
 - The cache-coherency and concurrency control protocols must work in coordination as well.
- Can be achieved easily by using shared memory parallel architecture Because single-processor multiuser systems themselves are designed to support concurrency control among transactions with the goal of increasing transaction throughput.
- However, difficult to achieve on shared-disk or shared-nothing architectures

Thank You !

